基于有限体积法的中药材热风烘干过程分析

摘　要

中药材热风烘干是药材加工的重要环节，内部升温、失水与收缩过程影响烘干效率及成品品质。本文将药材近似为轴对称圆柱体，依据烘房观测数据，依次建立预热阶段、变物性全程烘干及考虑径向收缩的热质传递模型，并采用有限体积法、后向 Euler 格式、Thomas 追赶法和 Picard 迭代法，求解计算药材不同径向位置的温度和水分浓度，确定达到目标水分浓度所需的烘干时间，为烘干工艺的合理制定提供依据。

针对问题一，首先结合烘房环境数据的分段线性插值，建立**预热平衡阶段药材热质传递模型**。然后求解药材不同径向位置的温度和水分浓度。预热 30min 时，中心与表面温度分别为 **33.5770** °C 和 **36.7865** °C，表面水分浓度降至 **1.5103** kg/kg，而中心仍基本保持初始值，表明**预热平衡阶段**水分散失主要集中于表层，完整结果保存于 `result1.xlsx`。插值敏感性和数值离散收敛性检验表明，计算结果对**插值方法**具有较好的稳健性，**离散误差**得到有效控制。**二维轴对称复核**验证了一维径向模型对药材中部区域的适用性。

针对问题二，将问题一的预热阶段模型扩展至**恒温干燥阶段**。首先依据经验公式，描述药材密度、比热容和导热系数随水分浓度的变化，以及水分扩散系数对温度和水分浓度的依赖。然后采用 **PCHIP 插值**构造烘房温度和空气水分浓度函数，建立了**全烘干过程药材热质传递模型**。求解时采用 **Picard 迭代**，逐次更新物性参数、温度和水分浓度。烘干 3h 时，中心与表面温度分别为 **49.8493**°C 和 **49.9666**°C，水分浓度分别为 **1.7662**kg/kg 和 **1.0081**kg/kg，此时温度分布已趋于均匀，而水分浓度仍存在明显的径向差异，完整计算结果保存于 `result2.xlsx`。收敛性检验表明，Picard 迭代在 3 次内即可达到预设要求，说明温度与水分浓度的耦合求解能够有效收敛。

针对问题三，在问题二模型的基础上，以药材各处水分浓度均不高于 0.15kg/kg 为目标，建立以目标水分浓度为终止条件的全程烘干模型。前 4h 的烘房环境数据沿用 PCHIP 插值，并以末 2h 观测值的**平均值**对**后续温度和水分浓度条件**进行**延拓**。水分浓度沿径向由中心向表面递减，故以**中心水分浓度**降至目标值作为终止判据，并采用**局部二分法**确定烘干完成时刻。计算得到所需烘干时间为 **57.6231**h，此时药材各径向位置均满足目标水分浓度要求，完整计算结果保存于 `result3.xlsx`。敏感性分析表明，水分扩散系数以及烘房稳态温度与药材初温之差是影响烘干时间的主要因素。

针对问题四，在问题三模型的基础上考虑药材烘干过程中的**径向收缩**。利用 PCHIP 插值构造**半径函数**，引入材料坐标将收缩区域映射至固定区间，并按附录 4 的表达式更新物性参数，建立**考虑尺寸收缩的全程烘干热质传递模型**。计算表明，药材半径在烘干初期减小较快，水分浓度始终由中心向表面递减。考虑收缩后，药材所需烘干时间为 **51.1749**h，完整计算结果保存于 `result4.xlsx`。固定半径退化一致性和水分质量平衡检验分别支持了坐标变换的正确性与数值求解的守恒性。

关键词： 中药材热风烘干　有限体积法　后向 Euler 格式　干燥终点　径向收缩

一、 问题重述

1.1 问题背景

中药材干燥是保障其品质的关键环节。热风烘干操作便利、适用性强，是常用的中药材干燥方式。该过程通常包括预热平衡和恒温干燥两个阶段。预热平衡阶段，药材在热风作用下逐渐升温，内外温差减小，并伴随水分迁移。恒温干燥阶段，烘房工况趋于稳定，药材内部水分持续向表面扩散并排出，使含水率逐步降低。在中药材烘干过程中，若工艺参数设置不当，易导致干燥周期延长、成本增加，并影响药材品质。



图 **1** 中药材热风烘干现场图

本文以圆柱形中药材为研究对象，结合烘房温度、空气水分浓度等环境数据及药材物性参数，研究药材内部温度场和水分浓度场的动态分布规律。基于传热传质理论，建立温度与水分浓度的热质传递模型，刻画预热平衡、恒温干燥及尺寸收缩条件下的干燥过程。通过数值模拟得到不同时间、不同径向位置的温度和水分浓度分布，并预测烘干所需时间，为烘干工艺优化提供理论依据。

1.2 问题提出

本文由预热平衡阶段入手，逐步扩展至完整烘干过程，并考虑尺寸收缩的影响，以确定药材内部温度和水分浓度的变化规律及烘干时间。

问题一：药材在烘干初期处于预热平衡阶段，烘房温度和空气水分浓度随时间变化。根据药材初始状态、几何尺寸及相关热质参数，建立药材内部温度和水分浓度的热质传递模型，计算 1800 s 内不同径向位置处的温度和水分浓度，为后续建模提供基础。

问题二：完整烘干过程持续时间较长，预热平衡和恒温干燥阶段的工况不同，且药材的密度、比热容、导热系数和水分扩散系数会随温度和水分浓度变化。因此，需要引入经验公式，建立物性参数动态变化条件下的热质传递模型，模拟前 3 h 内不同径向位置处的温度和水分浓度分布。

问题三：以药材各处水分浓度低于 0.15 kg/kg 作为烘干完成标准，在问题二模型的基础上继续模拟水分浓度变化，确定达到烘干要求所需的时间。

问题四：前述模型假设药材尺寸不变，但实际烘干过程中药材会因水分流失发生收

缩，进而影响内部传热传质过程。需结合药材半径变化数据和新的经验公式，建立考虑尺寸变化的干燥模型，重新计算烘干时间，并分析尺寸收缩的影响。

二、 问题分析

2.1 问题一的分析

问题一本质上是预热阶段圆柱形药材内部的非稳态传热与水分扩散问题。考虑轴对称性并忽略端面影响，将药材简化为一维径向模型；依据附件 1 的观测数据插值确定各计算时刻的表面对流换热与传质条件。根据附录 2，密度、比热容、导热系数及对流系数取为常数，水分扩散系数则随局部水分浓度变化。据此，采用有限体积法和后向 Euler 格式离散控制方程，并通过 Picard 迭代处理水分扩散项的非线性，得到预热阶段各径向位置的温度和水分浓度变化规律。

2.2 问题二的分析

问题二在问题一的基础上，将研究范围扩展至整个烘干过程。随着药材水分逐渐散失，密度、比热容和导热系数随水分浓度改变，水分扩散系数还受温度影响，使传热与水分迁移相互耦合。计算中沿用有限体积法和后向 Euler 格式，通过联合 Picard 迭代更新相关参数、温度和水分浓度，直至两场迭代结果同时收敛，得到烘干过程中药材各径向位置的温度和水分浓度变化。

2.3 问题三的分析

问题三本质上是在问题二热质传递模型的基础上，确定药材达到目标水分浓度所需的烘干时间。附件 1 仅提供前 4 h 的环境数据，前期采用 PCHIP 插值确定烘房温度和空气水分浓度，后续取附件 1 末 2 h 观测数据的平均值作为稳定干燥阶段的环境条件。药材表面持续失水，内部水分由中心向表层迁移，中心位置的水分浓度下降最慢，故以其首次降至 0.15 kg/kg 的时刻作为烘干完成时刻。在持续求解温度和水分浓度的基础上，采用局部二分法确定烘干所需时间。

2.4 问题四的分析

问题四在问题三全程烘干模型的基础上，进一步考虑药材失水导致的径向收缩。由附件 2 可知，药材半径随烘干过程减小，若仍在固定物理坐标下计算，部分径向节点将在后期落至药材外部。为此，引入材料坐标，将收缩区域映射为固定计算区间，并将半径变化引入温度和水分浓度控制方程及表面对流边界条件。结合附录 4 给出的参数表达式，采用联合 Picard 迭代求解热质传递过程，并持续监测中心水分浓度，直至其降至目标值，从而确定考虑径向收缩后的烘干时长。

三、 模型假设

1. 药材视为均质连续介质，初始温度和水分浓度在截面上均匀分布
2. 将药材近似为轴对称圆柱体，轴向温度与水分浓度梯度较小，主要考虑径向传热和传质过程
3. 烘房空气在药材表面分布均匀，其温度和水分浓度仅随时间变化
4. 药材与空气之间仅发生对流换热和对流传质，忽略辐射影响
5. 假设药材内部水分无生成或消耗，药材与烘房空气之间的水分交换满足质量守恒

四、 符号说明

符号	含义	单位
$T(r,t)$	药材内部温度场	°C
$C(r,t)$	药材内部水分浓度场	kg/kg
$T_a(t)$	烘房空气温度	°C
$C_a(t)$	烘房空气水分浓度	kg/kg
r	距药材中心的径向距离	m
t	烘干时间	s
R	药材初始半径	m
q_r	径向导热通量	$\mathrm{W/m^2}$
j_r	径向水分扩散通量	m/s
$\rho(C)$	药材密度	$\mathrm{kg/m^3}$
$c_p(C)$	药材比热容	$\mathrm{J/(kg\cdot K)}$
$k(C)$	药材导热系数	$\mathrm{W/(m\cdot K)}$
h	药材表面对流换热系数	$\mathrm{W/(m^2\cdot K)}$
h_m	药材表面对流传质系数	m/s
$D(C)$	药材内部水分扩散系数	$\mathrm{m^2/s}$
N_r	径向计算区间数	无量纲
$G(t)$	水分浓度阈值函数	kg/kg
t_{d}	药材达到目标水分浓度的烘干时刻	h
$R(t)$	考虑收缩时药材在时刻 t 的半径	m
ξ	材料坐标，$\xi = r/R(t)$	无量纲

五、 模型的建立与求解

5.1 问题一的模型建立与求解

5.1.1 问题一的模型准备

问题一研究药材在预热平衡阶段的温度和水分浓度变化。药材长 0.25 m、半径为 0.02 m，初始温度和水分浓度分别为 28 °C 和 2.55 kg/kg。设 $T(r,t)$、$C(r,t)$ 为药材内部温度和水分浓度，$T_a(t)$、$C_a(t)$ 为烘房空气温度和水分浓度。热量由空气经药材表面向内部传递，水分则由内部扩散至表面并逸入空气，如图 2所示。

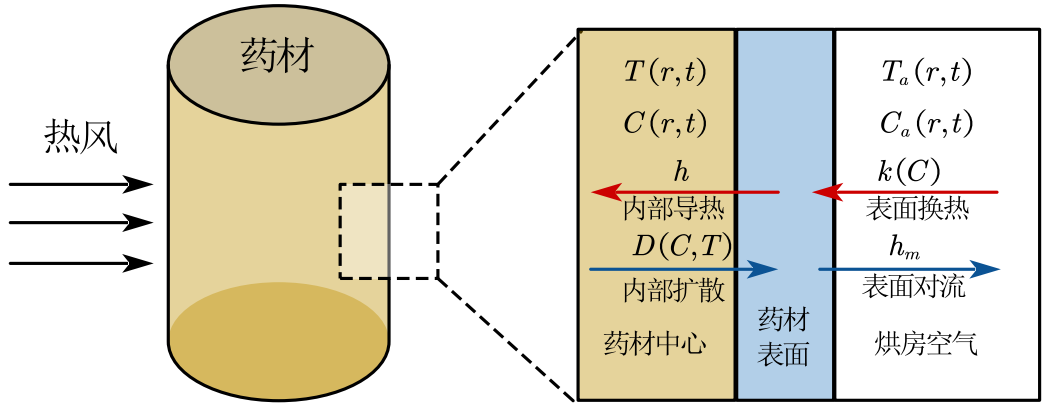


图 **2**　药材与烘房空气的换热换质示意图

附件 1 每隔 60, s 给出一组烘房环境数据，由于表面换热和传质通量随烘房温度、空气水分浓度变化，数值计算需在各计算时刻更新表面边界条件。因而采用分段线性插值构造 $T_a(t)$ 和 $C_a(t)$。并在各计算时刻更新表面对流边界条件。对任意 $t_i \le t \le t_{i+1}$，令 $X_a(t)$ 表示上述任一环境函数，则有

$$X_a(t) = X_{a,i} + \frac{t - t_i}{t_{i+1} - t_i}\left(X_{a,i+1} - X_{a,i}\right) \tag{1}$$

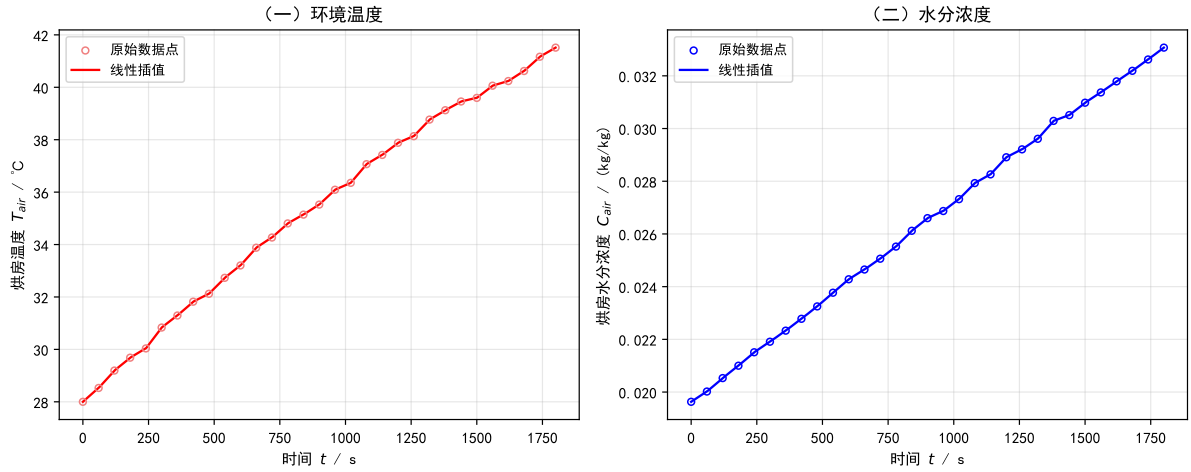


图 **3**　附件一线性插值

由此可得到 1800s 内各计算时刻对应的烘房温度和空气水分浓度，如图3所示，并将其作为药材表面对流换热和传质边界条件的输入。

5.1.2 问题一的模型建立

本问题研究预热平衡阶段药材内部温度和水分浓度的动态变化。结合题目给出的药材初始状态、几何尺寸、热质参数及烘房温湿环境数据，分析药材内部导热、水分扩散以及表面与空气之间的对流换热和传质过程，从而建立药材预热平衡阶段的热质传递模型。

温度场模型

药材内部热量沿径向传递，其导热通量满足傅里叶定律：

$$q_r = -k\frac{\partial T}{\partial r} \tag{2}$$

取内半径为 r、厚度为 $\mathrm{d}r$、长度为 L 的圆柱薄环微元，其内、外表面积及体积分别为

$$A_r = 2\pi r L, \qquad A_{r+\mathrm{d}r} = 2\pi(r+\mathrm{d}r)L, \tag{3}$$

$$\mathrm{d}V = \pi\left[(r+\mathrm{d}r)^2 - r^2\right]L = 2\pi r L\,\mathrm{d}r + o(\mathrm{d}r) \tag{4}$$

微元内热量积累等于径向导热的净输入，因此有

$$\rho c_p \frac{\partial T}{\partial t}\left(2\pi r L\,\mathrm{d}r\right) = 2\pi r L q_r(r) - 2\pi(r+\mathrm{d}r)L q_r(r+\mathrm{d}r) \tag{5}$$

两边除以 $2\pi r L\,\mathrm{d}r$，并令 $\mathrm{d}r \to 0$，得到局部能量守恒式：

$$\rho c_p \frac{\partial T}{\partial t} = -\frac{1}{r}\frac{\partial\left(r q_r\right)}{\partial r} \tag{6}$$

代入傅里叶定律，可得圆柱坐标下的温度场控制方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(kr\frac{\partial T}{\partial r}\right), \qquad 0 < r < R, \quad 0 < t \le 1800\,\mathrm{s} \tag{7}$$

上述方程在 $r = 0$ 处含有 $1/r$ 项。考虑药材的轴对称性，轴心位置不直接代入该式，而取极限处理：

$$\lim_{r\to 0}\frac{1}{r}\frac{\partial}{\partial r}\left(kr\frac{\partial T}{\partial r}\right) = 2k\frac{\partial^2 T}{\partial r^2}. \tag{8}$$

药材初始温度均匀分布：

$$T(r,0) = 28\,^{\circ}\mathrm{C}, \qquad 0 \le r \le R \tag{9}$$

中心处满足轴对称边界条件：

$$\left.\frac{\partial T}{\partial r}\right|_{r=0}=0,\qquad 0<t\le 1800\,\mathrm{s} \tag{10}$$

药材表面处，空气输入药材的对流热通量等于药材表面向内的导热通量，因此满足第三类边界条件：

$$k\left.\frac{\partial T}{\partial r}\right|_{r=R}=h\left[T_a(t)-T(R,t)\right],\qquad 0<t\le 1800\,\mathrm{s} \tag{11}$$

其中，$R=0.02\,\mathrm{m}$，$\rho=820\,\mathrm{kg/m^3}$，$c_p=2600\,\mathrm{J/(kg\cdot K)}$，$k=0.36\,\mathrm{W/(m\cdot K)}$，$h=25\,\mathrm{W/(m^2\cdot K)}$。

水分浓度场模型

以单位干物质为基准描述药材水分浓度 C。药材内部水分沿浓度降低方向扩散，其径向扩散通量满足菲克定律：

$$j_r=-D(C)\frac{\partial C}{\partial r} \tag{12}$$

对上述圆柱薄环微元，单位时间内水分浓度的累积等于径向水分通量的净输入：

$$\frac{\partial C}{\partial t}\left(2\pi r L\,\mathrm{d}r\right)=2\pi r L j_r(r)-2\pi(r+\mathrm{d}r)L j_r(r+\mathrm{d}r) \tag{13}$$

两边除以 $2\pi r L\,\mathrm{d}r$，并令 $\mathrm{d}r\to 0$，得到局部水分守恒式：

$$\frac{\partial C}{\partial t}=-\frac{1}{r}\frac{\partial\left(r j_r\right)}{\partial r} \tag{14}$$

代入菲克定律，得到圆柱坐标下的水分浓度控制方程：

$$\frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left[rD(C)\frac{\partial C}{\partial r}\right],\qquad 0<r<R,\quad 0<t\le 1800\,\mathrm{s} \tag{15}$$

水分扩散系数随局部水分浓度变化，故应保留在空间微分算子内，其表达式为

$$D(C)=7\times 10^{-9}e^{-\frac{0.89}{C}} \tag{16}$$

药材初始水分浓度均匀分布：

$$C(r,0)=2.55\,\mathrm{kg/kg},\qquad 0\le r\le R \tag{17}$$

中心处水分迁移满足对称边界条件：

$$\left.\frac{\partial C}{\partial r}\right|_{r=0}=0,\qquad 0<t\le 1800\,\mathrm{s} \tag{18}$$

药材表面向外的水分扩散通量等于空气侧对流传质通量，即

$$-D\big(C(R,t)\big)\left.\frac{\partial C}{\partial r}\right|_{r=R}=h_m\left[C(R,t)-C_a(t)\right],\qquad 0<t\le 1800\,\mathrm{s} \tag{19}$$

其中，$h_m=8\times10^{-7}\,\mathrm{m/s}$。

综上所述，在本问题中建立了**预热平衡阶段药材热质传递模型：**

$$\begin{cases}\text{温度传递模型} & \begin{cases}\rho c_p\dfrac{\partial T}{\partial t}=\dfrac{1}{r}\dfrac{\partial}{\partial r}\left(kr\dfrac{\partial T}{\partial r}\right)\\ T(r,0)=28\\ \left.\dfrac{\partial T}{\partial r}\right|_{r=0}=0\\ \left.k\,\dfrac{\partial T}{\partial r}\right|_{r=R}=h\left[T_a(t)-T(R,t)\right]\end{cases}\\ \text{水分迁移模型} & \begin{cases}\dfrac{\partial C}{\partial t}=\dfrac{1}{r}\dfrac{\partial}{\partial r}\left[rD(C)\dfrac{\partial C}{\partial r}\right]\\ C(r,0)=2.55\\ \left.\dfrac{\partial C}{\partial r}\right|_{r=0}=0\\ \left.-D\big(C(R,t)\big)\,\dfrac{\partial C}{\partial r}\right|_{r=R}=h_m\left[C(R,t)-C_a(t)\right]\end{cases}\end{cases} \tag{20}$$

5.1.3 问题一的模型求解

在本问题中，将药材预热平衡阶段的求解过程划分为烘房边界条件构造、温度场求解和水分浓度场求解三个部分。通过对各时间步的温度和水分浓度进行迭代计算，得到药材内部不同径向位置处的温度和水分浓度分布。

算法 1：预热平衡阶段药材热质传递模型求解

输入：烘房温度 $T_a(t)$、空气水分浓度 $C_a(t)$、药材初始条件及热质参数

输出：0 至 1800s 内药材不同径向位置处的温度和水分浓度。

Step 1 划分计算网格。将药材半径方向划分为 $N_r=800$ 个等距区间，取 $\Delta r=2.5\times10^{-5}\,\mathrm{m}$、$\Delta t=0.25\,\mathrm{s}$，记

$$r_i=i\Delta r,\qquad t^n=n\Delta t \tag{21}$$

Step 2 初始化温度和水分浓度。根据初始条件，令

$$T_i^0=28\,^\circ\mathrm{C},\qquad C_i^0=2.55\,\mathrm{kg/kg} \tag{22}$$

Step 3 求解温度场。由插值函数确定 $T_a(t^{n+1})$，采用有限体积法和后向 Euler 格式离散

温度控制方程，并代入中心对称边界和表面对流换热边界，得到

$$\boldsymbol{A}_T\boldsymbol{T}^{n+1}=\boldsymbol{T}^{n}+\boldsymbol{b}_T\left(T_a(t^{n+1})\right) \tag{23}$$

采用 Thomas 追赶法求解 $\boldsymbol{T}^{n+1}$。

Step 4 求解水分浓度场。取 $\boldsymbol{C}^{(0)}=\boldsymbol{C}^{n}$。第 k 次 Picard 迭代中，按相邻节点的平均浓度更新界面扩散系数：

$$D_{i+\frac{1}{2}}^{(k)}=7\times10^{-9}\mathrm{e}^{-\frac{0.89}{(C_i^{(k)}+C_{i+1}^{(k)})/2}} \tag{24}$$

代入中心对称边界、表面对流传质边界及 $C_a(t^{n+1})$，得到

$$\boldsymbol{A}_C\left(\boldsymbol{D}^{(k)}\right)\boldsymbol{C}^{(k+1)}=\boldsymbol{C}^{n}+\boldsymbol{b}_C\left(C_a(t^{n+1})\right) \tag{25}$$

采用 Thomas 追赶法求解 $\boldsymbol{C}^{(k+1)}$。

Step 5 判断迭代收敛性。若

$$\max_{0\le i\le N_r}\left|C_i^{(k+1)}-C_i^{(k)}\right|<10^{-12} \tag{26}$$

则取 $\boldsymbol{C}^{n+1}=\boldsymbol{C}^{(k+1)}$；否则返回 Step 4 继续迭代。

Step 6 保存当前计算结果。若 $t^{n+1}<1800\,\mathrm{s}$，则进入下一计算时刻并返回 Step 3；否则提取时间间隔为 1 s、径向间隔为 0.1 cm 的温度和水分浓度结果，保留四位小数。

上述算法的具体流程如图所示：

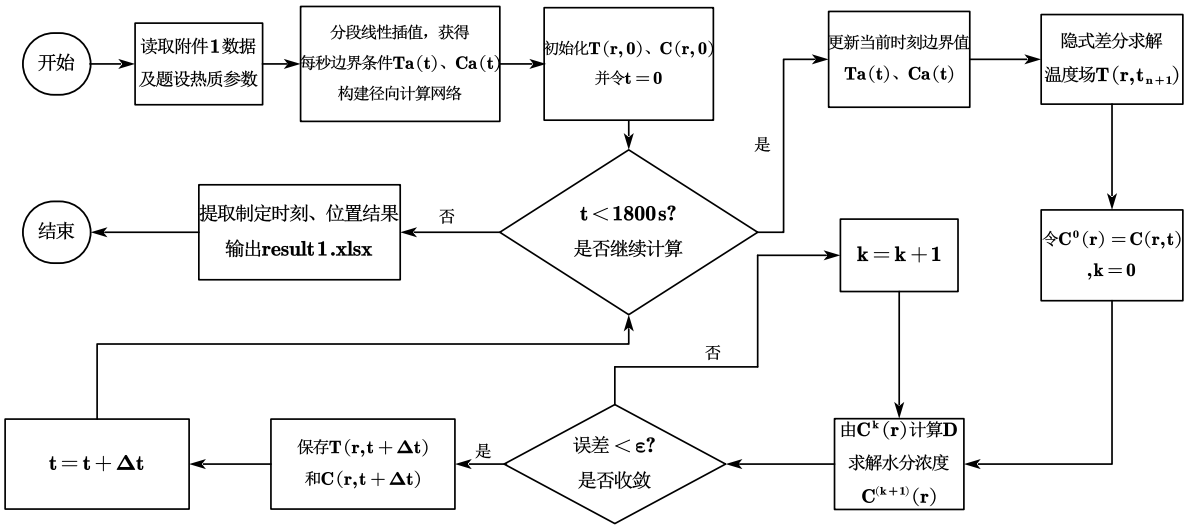


图 **4**　算法 **1** 流程图

该算法将药材半径方向进行等距离散，实现预热平衡阶段温度和水分浓度的逐时计算。由于水分扩散系数随浓度变化，在每个计算时刻内采用 Picard 迭代确定水分浓度分布。选取若干代表性时刻和径向位置，温度和水分浓度的计算结果分别如表 1和表 2所示。

表 **1**　**30** 分钟内药材的温度（°C）

时间 t/s	到药材中心的径向距离 r/cm				
	0	0.5	1.0	1.5	2.0
100	28.0001	28.0003	28.0039	28.0320	28.1797
300	28.0408	28.0636	28.1515	28.3675	28.8488
600	28.4537	28.5364	28.8042	29.3159	30.1643
900	29.3248	29.4588	29.8761	30.6167	31.7298
1200	30.5433	30.7104	31.2229	32.1133	33.4297
1500	31.9964	32.1874	32.7670	33.7479	35.1213
1800	33.5761	33.7727	34.3645	35.3624	36.7860

表 **2**　**30** 分钟内药材的水分浓度（kg/kg）

时间 t/s	到药材中心的径向距离 r/cm				
	0	0.5	1.0	1.5	2.0
100	2.5500	2.5500	2.5500	2.5500	2.2472
300	2.5500	2.5500	2.5500	2.5492	2.0509
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5103

5.1.4　问题一的模型结果分析

根据上述数值求解结果，绘制预热阶段药材内部温度和水分浓度的时空热力图，如图 5 所示。

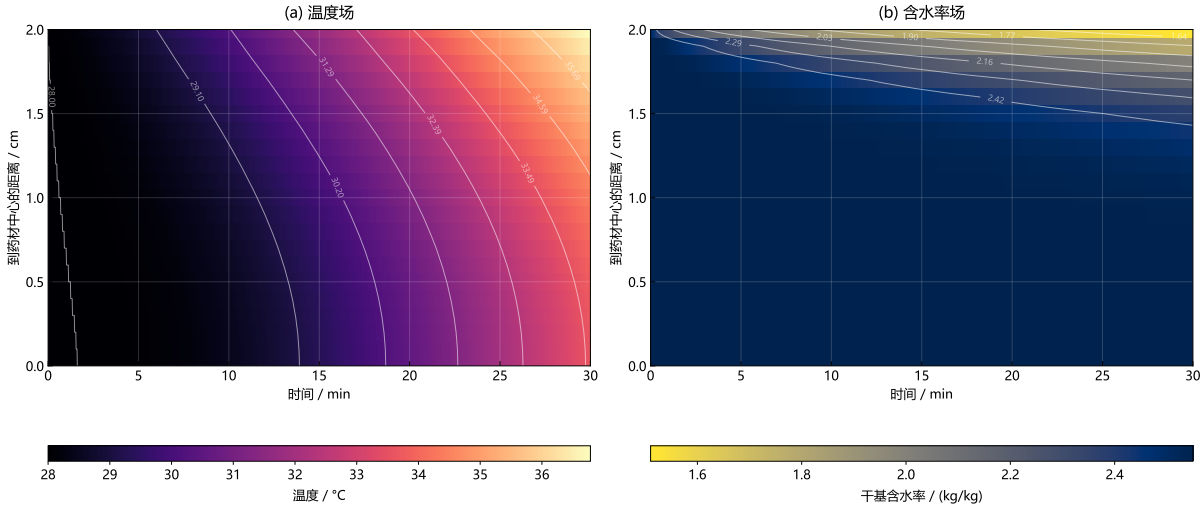


图 **5**　预热阶段药材内部温度和水分浓度的时空分布

由图 5可知，预热阶段等温线由表及里推进，药材表层升温快于中心。至 30 min 时，表面和中心温度分别为 36.7860 °C 和 33.5761 °C，温差约为 3.21 °C。水分散失主要集中于表层：表面水分浓度由 2.5500 kg/kg 降至 1.5103 kg/kg，距中心 1.5 cm 处为 2.3755 kg/kg，

中心基本保持初始值。可见，预热阶段以升温和表层失水为主，内部水分迁移尚不充分。

5.1.5 问题一的模型检验

插值方法敏感性检验

为考察环境数据插值方式对结果的影响，分别采用分段线性插值和 PCHIP 插值构造烘房温度及空气水分浓度函数，并在相同条件下进行求解，结果差异如图6所示。

由图6可知，温度差处于 $10^{-3}\,^{\circ}\mathrm{C}$ 量级，其中药材表面最大温差约为 $4.2\times10^{-3}\,^{\circ}\mathrm{C}$，水分浓度差处于 $10^{-6}\,\mathrm{kg/kg}$ 量级，其中表面最大差值约为 $1.9\times10^{-6}\,\mathrm{kg/kg}$，而中心处差异接近于零。综上，两种插值方法所得结果基本一致，表明模型对插值方式不敏感数值，结果具有较好的稳健性。

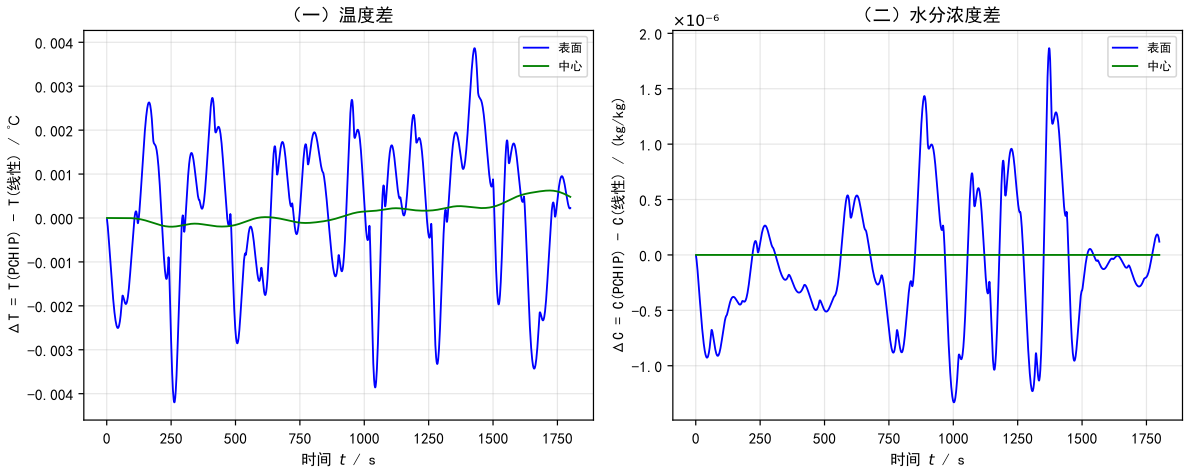


图 **6**　线性插值与 **PCHIP** 插值下药材的温度和水分浓度计算差异

数值离散收敛性检验

完整模型中水分扩散系数和表面环境条件均随时间变化，难以获得解析解。为检验数值离散格式，在保持圆柱几何和边界条件不变的基础上，将水分扩散系数取为常数，构造线性扩散验证问题。

该问题的径向特征函数为零阶第一类 Bessel 函数，水分浓度可表示为

$$C(r,t)=C_a+(C_0-C_a)\sum_{m=1}^{\infty}A_m J_0\left(\lambda_m\frac{r}{R}\right)e^{-\frac{\lambda_m^2 Dt}{R^2}} \tag{27}$$

其中，J_0 为零阶第一类 Bessel 函数，λ_m 为特征方程

$$\lambda_m J_1(\lambda_m)=\frac{h_m R}{D}J_0(\lambda_m) \tag{28}$$

的第 m 个正根，A_m 由初始条件确定。计算时取级数前 60 项作为解析参照解。记最大绝对误差为

$$E_\infty=\max_i\left|C_i^{\mathrm{num}}-C^{\mathrm{ana}}\left(r_i,t_{\mathrm{end}}\right)\right| \tag{29}$$

分别考察径向网格数和时间步长对误差的影响，结果如图 7 所示。

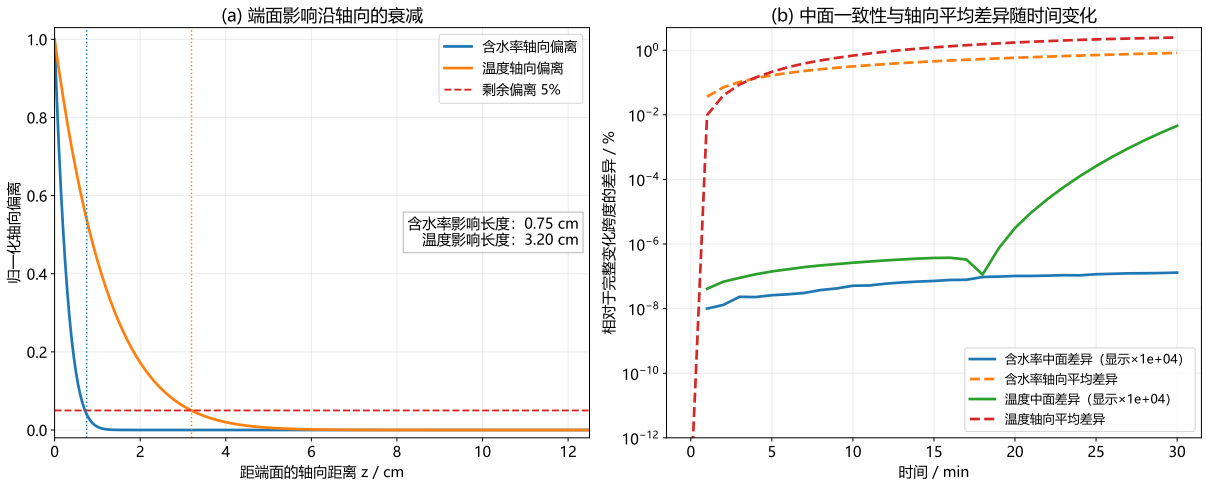


图 **8**　一维径向模型的二维轴对称复核

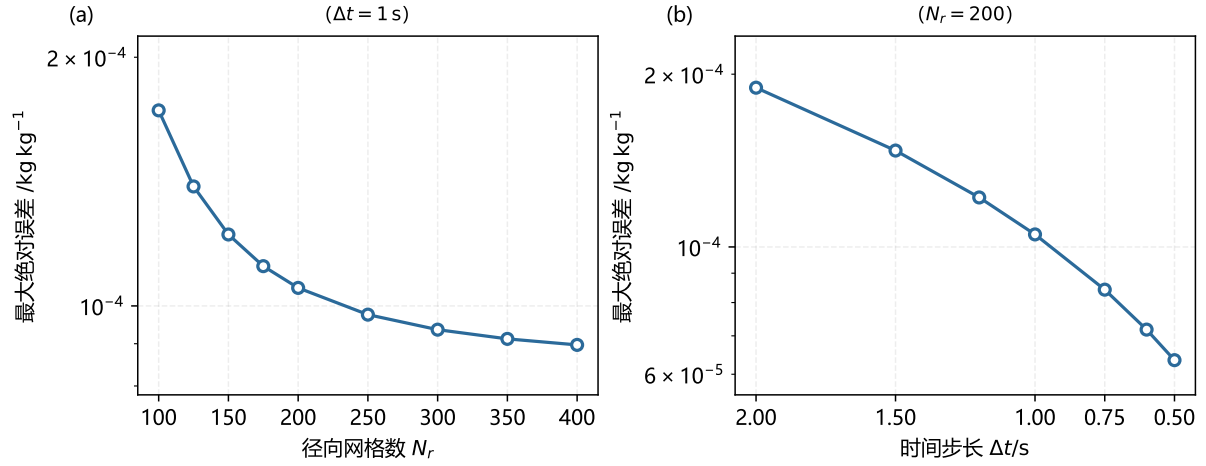


图 **7**　**Bessel** 解析解下网格与时间步长的收敛性检验

由图 7(a) 可知，在 $\Delta t = 1\,\mathrm{s}$ 时，网格数由 100 增加至 400，最大绝对误差由 $1.725 \times 10^{-4}\,\mathrm{kg/kg}$ 降至 $8.970 \times 10^{-5}\,\mathrm{kg/kg}$。由图 7(b) 可知，在 $N_r = 200$ 时，时间步长由 2 s 减小至 0.5 s，误差由 $1.895 \times 10^{-4}\,\mathrm{kg/kg}$ 降至 $6.349 \times 10^{-5}\,\mathrm{kg/kg}$。

因此，随着网格加密和时间步减小，数值解逐步逼近 Bessel 解析解。问题一正式计算采用 $N_r = 800$、$\Delta t = 0.25\,\mathrm{s}$，数值离散误差处于可控范围。

一维径向假设复核

为检验忽略端面传递对药材主体区域热质传递的影响，建立 r-z 二维轴对称参照模型。二维模型取半径 $R = 0.02\,\mathrm{m}$、半长 $H = 0.125\,\mathrm{m}$，侧面和端面施加对流换热、传质边界，中心轴和中面满足对称边界条件。除计入端面交换外，两模型采用相同的初始条件、物性参数、环境条件及数值方法。

比较二维模型中面处的径向分布与一维结果，并考察端部影响沿轴向的衰减，结果如图 8所示。由图 8(a) 可知，以轴向偏离衰减至端面偏离的 5% 为判据，水分浓度和温度的单侧端部影响长度分别约为 0.75 cm 和 3.20 cm。由图 8(b) 可知，二维模型中面处的温度和水分浓度径向分布与一维模型结果基本一致，而轴向平均结果存在一定偏差。

因此，端面效应主要局限于近端局部区域，药材中部及主体区域的轴向影响相对较

弱。对于本题关注的不同径向位置处温度和水分浓度演化，一维径向模型能够满足预热阶段热质传递过程的描述需求。

5.2 问题二的模型建立与求解

5.2.1 问题二的模型建立

问题二沿用问题一的轴对称圆柱模型及初始条件。与问题一不同，药材的密度、比热容和导热系数随水分浓度变化，水分扩散系数同时受水分浓度和温度影响，故需采用附录 3 中的经验公式对热质传递模型进行修正。

附录 3 中的水分扩散系数以绝对温度为自变量，为保证温标一致性，将药材内部温度换算为

$$T_{\mathrm{K}}(r,t) = T(r,t) + 273.15 \tag{30}$$

药材的密度、比热容、导热系数和水分扩散系数表示为

$$\begin{cases} \rho(C) = 650 + 128C \\ c_p(C) = 1450 + 2736\dfrac{C}{C+1} \\ k(C) = 0.21 + 0.38\dfrac{C}{C+1} \\ D\big(C, T_{\mathrm{K}}\big) = 2.4 \times 10^{-3} e^{-\frac{0.45}{C}} e^{-\frac{3850}{T_{\mathrm{K}}}} \end{cases} \tag{31}$$

温度场模型

在问题一微元能量守恒方程的基础上，将常数热物性参数替换为随局部水分浓度变化的参数，可得温度场控制方程：

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rk(C)\frac{\partial T}{\partial r}\right], \qquad 0 < r < R, \quad 0 < t \le 10800\,\mathrm{s} \tag{32}$$

药材初始温度和中心对称边界条件与问题一一致。表面对流换热边界修正为

$$k\big(C(R,t)\big)\left.\frac{\partial T}{\partial r}\right|_{r=R} = h\left[T_a(t) - T(R,t)\right], \qquad 0 < t \le 10800\,\mathrm{s} \tag{33}$$

水分浓度场模型

水分扩散系数随局部水分浓度和温度变化，故水分浓度控制方程修正为

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left[rD\big(C, T_{\mathrm{K}}\big)\frac{\partial C}{\partial r}\right], \qquad 0 < r < R, \quad 0 < t \le 10800\,\mathrm{s} \tag{34}$$

药材初始水分浓度和中心对称边界条件与问题一一致。表面对流传质边界修正为

$$-D\big(C(R,t), T_{\mathrm{K}}(R,t)\big)\left.\frac{\partial C}{\partial r}\right|_{r=R} = h_m\left[C(R,t) - C_a(t)\right], \qquad 0 < t \le 10800\,\mathrm{s} \tag{35}$$

其中，h 和 h_m 沿用问题一中的设定，$T_a(t)$ 和 $C_a(t)$ 由附件 1 数据经 PCHIP 插值得到。

综上所述，在本问题中建立了**全烘干过程药材热质传递模型：**

$$
\begin{cases}
\text{温度传递模型} & \begin{cases}
\rho(C)c_p(C)\dfrac{\partial T}{\partial t} = \dfrac{1}{r}\dfrac{\partial}{\partial r}\left[rk(C)\dfrac{\partial T}{\partial r}\right] \\
T(r,0) = 28 \\
\left.\dfrac{\partial T}{\partial r}\right|_{r=0} = 0 \\
k\big(C(R,t)\big)\left.\dfrac{\partial T}{\partial r}\right|_{r=R} = h\left[T_a(t) - T(R,t)\right]
\end{cases} \\
\text{水分迁移模型} & \begin{cases}
\dfrac{\partial C}{\partial t} = \dfrac{1}{r}\dfrac{\partial}{\partial r}\left[rD\big(C,T_{\mathrm{K}}\big)\dfrac{\partial C}{\partial r}\right] \\
C(r,0) = 2.55 \\
\left.\dfrac{\partial C}{\partial r}\right|_{r=0} = 0 \\
-D\big(C(R,t),T_{\mathrm{K}}(R,t)\big)\left.\dfrac{\partial C}{\partial r}\right|_{r=R} = h_m\left[C(R,t) - C_a(t)\right]
\end{cases}
\end{cases}
\tag{36}
$$

5.2.2 问题二的模型求解

问题二仍采用问题一中的有限体积离散、后向 Euler 隐式格式和 Thomas 追赶法。考虑到烘房温度和空气水分浓度仅以 60s 为间隔给出，且在实际烘干过程中连续变化，采用 PCHIP 插值构造相应的环境条件函数，以保持原始数据的变化趋势并使其在时间上平滑变化。

不同于问题一，药材的密度、比热容和导热系数随水分浓度变化，水分扩散系数同时受温度和水分浓度影响。因此，温度场与水分浓度场不能依次独立求解，需在每个计算时刻采用 Picard 迭代，交替更新物性参数、温度场和水分浓度场，直至迭代结果收敛。

算法 2：全烘干过程药材热质传递模型求解

输入：烘房温度 $T_a(t)$、空气水分浓度 $C_a(t)$、药材初始状态及附录 3 中的物性参数。

输出：0 至 10800 s 内药材不同径向位置处的温度和水分浓度。

Step 1 沿用算法 1 的径向计算网格和时间离散间隔，并采用 PCHIP 插值构造烘房温度函数 $T_a(t)$ 和空气水分浓度函数 $C_a(t)$。根据初始条件，令

$$
T_i^0 = 28\,^{\circ}\mathrm{C}, \qquad C_i^0 = 2.55\,\mathrm{kg/kg} \tag{37}
$$

除物性参数更新方式外，温度场和水分浓度场的空间离散、时间离散、边界条件处理及三对角线性方程组的 Thomas 追赶法求解均沿用算法 1。算法 2 进一步考虑热物性参数随水分浓度变化，以及水分扩散系数随温度和水分浓度变化所产生的热质耦合非线

性。

Step 2 对第 n 个计算时刻，计算时刻 t^{n+1} 对应的烘房温度 $T_a(t^{n+1})$ 和空气水分浓度 $C_a(t^{n+1})$，并以当前温度场和水分浓度场作为 Picard 迭代初值：

$$T_i^{(0)} = T_i^n, \qquad C_i^{(0)} = C_i^n \tag{38}$$

Step 3 在第 q 次迭代中，根据水分浓度场 $\boldsymbol{C}^{(q)}$ 更新密度、比热容和导热系数，并沿用算法 1 的离散和求解方法得到温度场 $\boldsymbol{T}^{(q+1)}$。随后，将温度场转换为绝对温度场 $\boldsymbol{T}_{\mathrm{K}}^{(q+1)}$，根据 $\boldsymbol{C}^{(q)}$ 和 $\boldsymbol{T}_{\mathrm{K}}^{(q+1)}$ 更新水分扩散系数，进而求得水分浓度场 $\boldsymbol{C}^{(q+1)}$。

Step 4 判断温度场和水分浓度场是否同时收敛。若满足

$$\max_i \left| T_i^{(q+1)} - T_i^{(q)} \right| < 10^{-6}, \qquad \max_i \left| C_i^{(q+1)} - C_i^{(q)} \right| < 10^{-8} \tag{39}$$

则取

$$\boldsymbol{T}^{n+1} = \boldsymbol{T}^{(q+1)}, \qquad \boldsymbol{C}^{n+1} = \boldsymbol{C}^{(q+1)} \tag{40}$$

否则，以第 $q+1$ 次迭代结果作为新的迭代初值，重复 Step 3 至 Step 4。

Step 5 将收敛后的温度场和水分浓度场更新至下一时间层，并令 $n \leftarrow n+1$。若

$$t^n < 10800\,\mathrm{s} \tag{41}$$

则返回 Step 2；否则结束计算。最后按 1 s 的时间间隔和 0.1 cm 的径向间隔输出结果，并提取题目规定时刻和径向位置处的数据，温度分布和水分浓度分布分别如表 3和表 4所示。

表 **3**　**3** 小时内药材的温度（°C）

时间 t/h	到药材中心的径向距离 r/cm				
	0	0.5	1.0	1.5	2.0
0.5	32.1911	32.3839	32.9674	33.9620	35.4141
1.0	40.3822	40.5542	41.0609	41.8784	43.0001
1.5	45.8464	45.9345	46.1927	46.6049	47.1393
2.0	48.4498	48.4876	48.5980	48.7732	49.0030
2.5	49.4667	49.4789	49.5133	49.5645	49.6617
3.0	49.8493	49.8552	49.8746	49.9100	49.9666

表 **4**　**3** 小时内药材的水分浓度（kg/kg）

时间 t/h	到药材中心的径向距离 r/cm				
	0	0.5	1.0	1.5	2.0
0.5	2.5499	2.5489	2.5255	2.3257	1.6487
1.0	2.5257	2.4947	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6260	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1167
3.0	1.7662	1.7166	1.5702	1.3333	1.0081

5.2.3　问题二的结果分析

根据全烘干过程的数值求解结果，绘制药材内部温度场和水分浓度场的时空热力图，如图 9 所示。

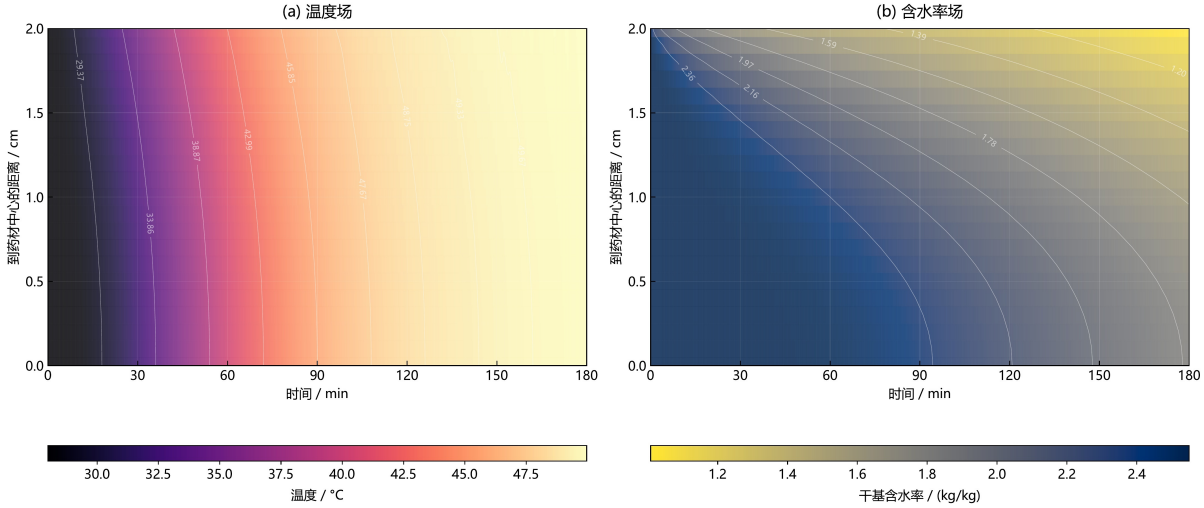


图 **9**　全烘干过程药材内部温度和水分浓度的时空分布

由图 9及表 3、表 4可知，药材表层在早期升温和失水更快。0.5h 时，表面与中心温度相差约 3.22°C，水分浓度分别为 1.6487kg/kg 和 2.5499kg/kg。至 3.0h 时，温度场已基本均匀，而中心水分浓度仍高于表面，表明后期干燥主要受内部水分迁移制约。

综上，全烘干过程可概括为药材快速升温和水分持续扩散两个阶段。温度场较快趋于均匀，而水分浓度梯度在整个过程中持续存在，后期干燥速率主要受药材内部水分向表面的迁移过程制约。

5.2.4　问题二的模型检验

问题二中，药材热物性参数随水分浓度变化，水分扩散系数同时受温度和水分浓度影响，温度场与水分浓度场存在耦合关系。因此，在各计算时刻采用联合 Picard 迭代更新温度场、水分浓度场及相关物性参数。选取 0.5h、1.5h 和 3.0h 三个代表性时刻，记录相邻两次迭代结果的最大差值，以检验迭代过程的收敛性。

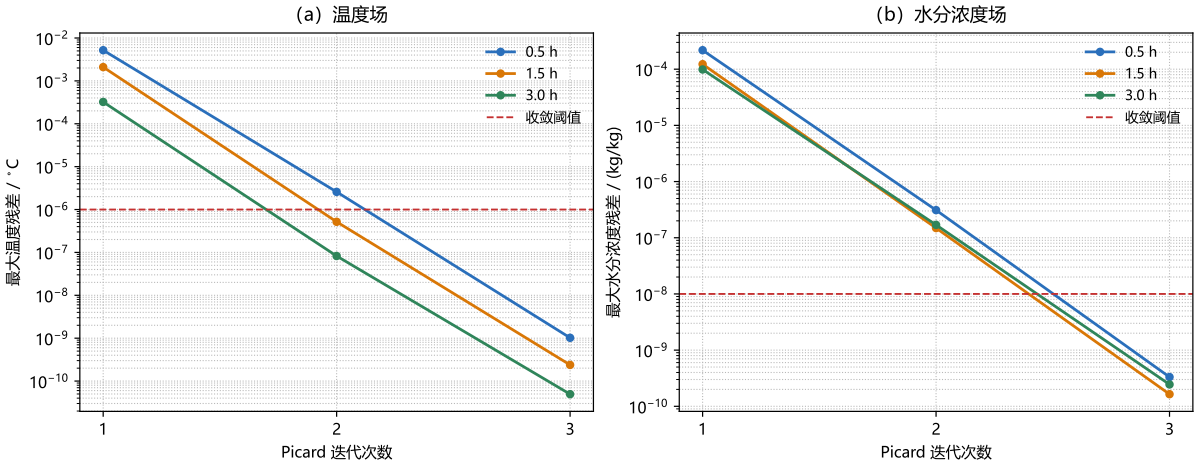


图 **10**　联合 **Picard** 迭代收敛性检验

由图 10可知，三个代表性时刻下的温度残差和水分浓度残差均随迭代次数迅速下降，并在 3 次迭代内分别满足 $10^{-6}\,^{\circ}\mathrm{C}$ 和 $10^{-8}\mathrm{kg/kg}$ 的收敛要求。这说明温度场、水分浓度场及相关物性参数能够实现协调更新，所采用的联合 Picard 迭代具有良好的收敛性和数值稳定性。

5.3　问题三的模型建立与求解

5.3.1　问题三的模型建立

问题三在问题二热质传递模型的基础上，确定药材达到目标水分浓度所需的烘干时间。药材内部热质传递机理、物性参数表达式、初始条件及表面对流换热和传质边界均沿用问题二。本问的重点由求解指定时段内的温度场和水分浓度场，转变为确定药材首次整体达到干燥要求的时刻。

环境条件延拓

附件 1 的环境数据覆盖烘干开始后前 4 h。在该时段内，采用 PCHIP 插值构造烘房温度和空气水分浓度函数。考虑到附件末段环境条件已趋于稳定，取最后 2 h 内各观测值的算术平均作为稳定干燥阶段的环境条件。

记最后 2 h 内的观测时刻集合为

$$\mathcal{I}=\left\{t_j \mid 7200\,\mathrm{s}\le t_j\le 14400\,\mathrm{s}\right\} \tag{42}$$

其中，N_{p} 为集合 $\mathcal{I}$ 中的观测值数量，则有

$$\overline{T}_a=\frac{1}{N_{\mathrm{p}}}\sum_{t_j\in\mathcal{I}}T_a(t_j)=49.97\,^{\circ}\mathrm{C} \tag{43}$$

$$\overline{C}_a=\frac{1}{N_{\mathrm{p}}}\sum_{t_j\in\mathcal{I}}C_a(t_j)=0.0499\,\mathrm{kg/kg} \tag{44}$$

因此，烘房环境条件表示为

$$T_a(t) = \begin{cases} T_{a,\mathrm{PCHIP}}(t), & 0 \le t \le 14400\,\mathrm{s} \\ \overline{T}_a, & t > 14400\,\mathrm{s} \end{cases} \tag{45}$$

$$C_a(t) = \begin{cases} C_{a,\mathrm{PCHIP}}(t), & 0 \le t \le 14400\,\mathrm{s} \\ \overline{C}_a, & t > 14400\,\mathrm{s} \end{cases} \tag{46}$$

烘干终止判据

题目要求药材各处水分浓度均低于 $0.15\,\mathrm{kg/kg}$。为保证判定结果覆盖药材整个径向截面，定义时刻 t 的最大水分浓度为

$$C_{\max}(t) = \max_{0 \le r \le R} C(r,t). \tag{47}$$

进一步定义阈值函数

$$G(t) = C_{\max}(t) - 0.15. \tag{48}$$

当 $G(t) \ge 0$ 时，药材内部仍存在水分浓度不低于目标值的位置；当 $G(t) < 0$ 时，药材各径向位置的水分浓度均低于目标值。因此，将阈值函数首次达到零的时刻定义为烘干完成的临界时刻：

$$t_{\mathrm{d}} = \min\{t > 0 \mid G(t) < 0\} \tag{49}$$

药材初始水分浓度沿径向均匀分布，中心处满足对称边界条件

$$\frac{\partial C}{\partial r}(0,t) = 0 \tag{50}$$

在药材表面，水分向空气迁移，且烘干过程中通常有 $C(R,t) > C_a(t)$。由表面对流传质边界条件

$$-D\big(C(R,t), T_{\mathrm{K}}(R,t)\big)\frac{\partial C}{\partial r}(R,t) = h_m\big[C(R,t) - C_a(t)\big] \tag{51}$$

可知，因 $D > 0$、$h_m > 0$，故有

$$\frac{\partial C}{\partial r}(R,t) < 0 \tag{52}$$

对于初始均匀、轴对称且扩散系数为正的径向扩散过程，在中心对称边界和表面失水边界共同作用下，水分浓度沿径向保持由中心向表面递减，即

$$C(0,t) \ge C(r,t), \qquad 0 \le r \le R \tag{53}$$

因此，药材截面内的最大水分浓度始终位于中心位置：

$$C_{\max}(t) = C(0,t) \tag{54}$$

故烘干完成时刻可进一步写为

$$t_{\mathrm{d}} = \min\{t > 0 \mid C(0,t) < 0.15\} \tag{55}$$

烘干完成时刻定位

当相邻两个计算时刻满足

$$G(t^n) > 0, \qquad G(t^{n+1}) \le 0 \tag{56}$$

则烘干完成时刻位于区间 $[t^n, t^{n+1}]$ 内。记该区间左右端点为 t_{L} 和 t_{R}，取中点

$$t_{\mathrm{M}} = \frac{t_{\mathrm{L}} + t_{\mathrm{R}}}{2} \tag{57}$$

以 t_{L} 时刻的温度场和水分浓度场为初始状态，重新求解至 t_{M}。若 $G(t_{\mathrm{M}}) > 0$，则以 t_{M} 更新左端点；否则，以 t_{M} 更新右端点。重复上述过程，直至

$$t_{\mathrm{R}} - t_{\mathrm{L}} < \varepsilon_t, \qquad \varepsilon_t = 0.01\,\mathrm{s} \tag{58}$$

最终取

$$t_{\mathrm{d}} = \frac{t_{\mathrm{L}} + t_{\mathrm{R}}}{2} \tag{59}$$

作为药材烘干完成时刻。

综上所述，在本问题中建立了**以目标水分浓度为终止条件的全程烘干热质传递模型：**

$$
\begin{cases}
\text{温度传递模型} & \begin{cases}
\rho(C)c_p(C)\dfrac{\partial T}{\partial t} = \dfrac{1}{r}\dfrac{\partial}{\partial r}\left[rk(C)\dfrac{\partial T}{\partial r}\right] \\
T(r,0) = 28 \\
\dfrac{\partial T}{\partial r}(0,t) = 0 \\
k\big(C(R,t)\big)\dfrac{\partial T}{\partial r}(R,t) = h\big[T_a(t) - T(R,t)\big]
\end{cases} \\
\text{水分迁移模型} & \begin{cases}
\dfrac{\partial C}{\partial t} = \dfrac{1}{r}\dfrac{\partial}{\partial r}\left[rD\big(C,T_{\mathrm{K}}\big)\dfrac{\partial C}{\partial r}\right] \\
C(r,0) = 2.55 \\
\dfrac{\partial C}{\partial r}(0,t) = 0 \\
-D\big(C(R,t),T_{\mathrm{K}}(R,t)\big)\dfrac{\partial C}{\partial r}(R,t) = h_m\big[C(R,t) - C_a(t)\big]
\end{cases} \\
\text{烘干终止条件} & \Big\{t_{\mathrm{d}} = \min\{t > 0 \mid C(0,t) < 0.15\}
\end{cases}
\tag{60}
$$

5.3.2 问题三的模型求解

问题三仍采用问题二中的有限体积离散、后向 Euler 隐式格式、Thomas 追赶法和联合 Picard 迭代。与问题二不同的是，计算不在预设时刻结束，而是持续推进至药材中心水分浓度首次达到目标值，并通过局部二分法精确定位烘干完成时刻。

算法 3：以目标水分浓度为终止条件的全程烘干模型求解

输入：问题二热质传递模型、烘房环境函数、稳定干燥阶段环境参数及药材初始状态。

输出：药材烘干所需时间 t_{d} 及烘干过程中的水分浓度结果。

Step 1 将药材半径方向划分为 $N_r = 200$ 个等距区间，取空间步长 $\Delta r = 1.0 \times 10^{-4}\,\mathrm{m}$、时间离散间隔 $\Delta t = 60\,\mathrm{s}$，记

$$
r_i = i\Delta r, \qquad t^n = n\Delta t \tag{61}
$$

根据初始条件，令

$$
T_i^0 = 28\,^{\circ}\mathrm{C}, \qquad C_i^0 = 2.55\,\mathrm{kg/kg} \tag{62}
$$

Step 2 在第 n 个时间层，根据前文构造的烘房环境函数确定时刻 t^{n+1} 的边界条件。沿用算法 2 的联合 Picard 迭代过程，求解温度场和水分浓度场；迭代满足

$$
\max_i \left|T_i^{(q+1)} - T_i^{(q)}\right| < 10^{-6}, \qquad \max_i \left|C_i^{(q+1)} - C_i^{(q)}\right| < 10^{-8} \tag{63}
$$

后，得到

$$\boldsymbol{T}^{n+1}, \qquad \boldsymbol{C}^{n+1} \tag{64}$$

Step 3 计算时刻 t^{n+1} 的阈值函数 $G(t^{n+1})$。若

$$G(t^{n+1}) > 0 \tag{65}$$

则药材尚未达到烘干要求，保存当前结果，令 $n \leftarrow n+1$，并返回 Step 2。

若

$$G(t^{n+1}) \leq 0 \tag{66}$$

则烘干完成时刻位于区间 $[t^n, t^{n+1}]$ 内，进入 Step 4。

Step 4 按前文所述的局部二分法定位烘干完成时刻。

Step 5 输出每隔 60 s、每隔 0.1 cm 的水分浓度结果，并提取每隔 6 h 及烘干完成时刻、距药材中心 0、0.5、1.0、1.5 和 2.0 cm 处的水分浓度。

药材烘干过程中不同径向位置的水分浓度变化如表 5所示。

表 **5**　药材烘干过程的水分浓度（kg/kg）

时间 t/h	距药材中心的径向距离 r/cm				
	0	0.5	1.0	1.5	2.0
6.0	1.0188	0.9899	0.9031	0.7563	0.5336
12.0	0.4573	0.4443	0.4041	0.3305	0.1647
18.0	0.2994	0.2921	0.2692	0.2257	0.0845
24.0	0.2382	0.2331	0.2171	0.1858	0.0664
30.0	0.2061	0.2021	0.1894	0.1643	0.0598
36.0	0.1861	0.1828	0.1720	0.1506	0.0566
42.0	0.1723	0.1693	0.1599	0.1409	0.0548
48.0	0.1620	0.1594	0.1509	0.1336	0.0536
54.0	0.1540	0.1516	0.1438	0.1279	0.0529
57.6231	0.1500	0.1477	0.1402	0.1250	0.0526

由表 5可知，烘干至 57.6231 h 时，药材中心水分浓度降至 0.1500 kg/kg，其余径向位置均低于目标值，满足药材各处的干燥要求。因此，不考虑径向收缩时，药材所需烘干时间为 **57.6231** h。

5.3.3　问题三的结果分析

为直观展示全程烘干过程中药材内部水分浓度的时空演化特征，将各计算时刻、各径向位置的数值结果绘制为时空热力图，并叠加目标水分浓度 0.15 kg/kg 的等浓度线，如图 11所示。

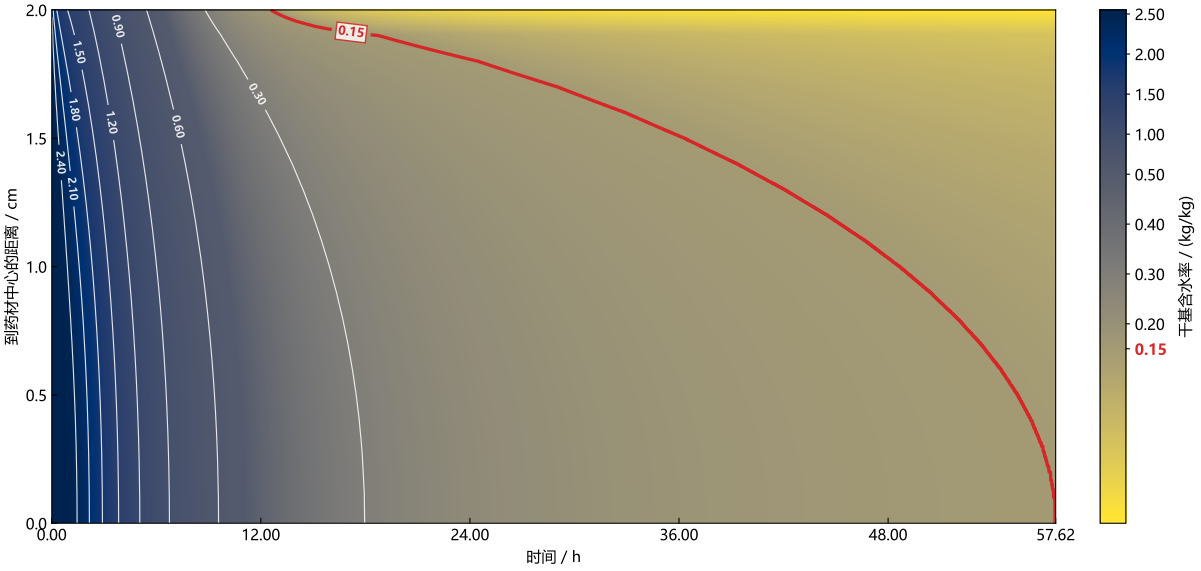


图 **11**　全烘干过程药材内部水分浓度的时空分布

由图 11及表 5可知，药材水分浓度随烘干过程持续下降，并始终由中心向表面递减。烘干 6.0 h 时，中心与表面水分浓度分别为 1.0188 kg/kg 和 0.5336 kg/kg，径向梯度已较为明显。此后，0.15 kg/kg 等浓度线逐步向中心推进；至 54.0 h 时，中心浓度仍为 0.1540 kg/kg。中心于 57.6231 h 最后达到目标值，表明后期干燥主要受内部水分向表面的迁移过程制约。

5.3.4　问题三的模型检验

为考察物性参数和边界参数的不确定性对烘干时间预测结果的影响，采用单因素扰动法进行敏感性检验。基准计算中，前 4 h 的烘房环境采用 PCHIP 插值，后续稳定阶段温度和空气水分浓度取附件 1 末 2 h 观测值的平均值，分别为 49.9685 °C 和 0.049865 kg/kg，对应基准烘干时间为 57.6231 h，计算结果如表 6所示。

表 **6**　物性与边界参数对烘干完成时间的敏感性

扰动参数	-5% 时 t_{d}/h	$+5\%$ 时 t_{d}/h	敏感系数 S_p
水分扩散系数 D	60.3106	55.2017	-0.886572
恒温段温差 $T_\infty - T_0$	59.6881	55.6615	-0.698747
传质系数 h_m	57.9671	57.3228	-0.111801
空气水分浓度 C_∞	57.5918	57.6665	0.012959
密度 ρ	57.6208	57.6299	0.001588
比热容 c_p	57.6208	57.6299	0.001588
换热系数 h	57.6295	57.6216	-0.001357
导热系数 k	57.6261	57.6247	-0.000235

其中，

$$S_p = \frac{t_{\mathrm{d}}\big(p_0(1+\delta)\big) - t_{\mathrm{d}}\big(p_0(1-\delta)\big)}{2\delta t_{\mathrm{d},0}}, \qquad \delta = 0.05 \tag{67}$$

$S_p < 0$ 表示参数增大使烘干时间缩短，$S_p > 0$ 表示参数增大使烘干时间延长。

由表 6可知，水分扩散系数 D 和恒温段温差 $T_\infty - T_0$ 对烘干时间的影响最为显著，传质系数 h_m 次之，其余参数在 $\pm 5\%$ 扰动下引起的烘干时间变化较小。各参数变化引起的烘干时间变化方向与热质传递机理一致，即增大扩散系数、温差或传质系数均会缩短烘干时间，而增大空气水分浓度会延长烘干时间。该检验表明，模型能够合理反映主要参数变化对烘干过程的影响，且对密度、比热容、导热系数和换热系数的小幅扰动具有一定稳健性。

5.4 问题四的模型建立与求解

5.4.1 问题四的模型建立

问题四在问题三热质传递模型的基础上，进一步考虑药材在烘干过程中的径向收缩。药材半径由附件 2 给出，采用 PCHIP 插值构造半径函数 $R(t)$。设附件 2 中最后一个观测时刻为 t_R，对应半径为 $R_{\min}$，则在观测区间内有

$$R(t) = R_{\mathrm{PCHIP}}(t), \qquad 0 \le t \le t_R \tag{68}$$

当 $t > t_R$ 时，药材半径保持为

$$R(t) = R_{\min} \tag{69}$$

坐标变换

药材半径随时间变化，直接在物理坐标 r 下求解将导致计算区域不断收缩。为此，引入材料坐标

$$\xi = \frac{r}{R(t)}, \qquad 0 \le \xi \le 1 \tag{70}$$

相应地，材料点在物理空间中的径向位置为

$$r = \xi R(t) \tag{71}$$

其中，$\xi = 0$ 和 $\xi = 1$ 分别表示药材中心和表面。假设药材沿径向均匀收缩，则材料点的相对位置 ξ 在烘干过程中保持不变。由于水分浓度以干基计量，材料单元内干物质质量守恒，故在材料坐标下无需额外引入由边界收缩导致的对流项。

烘房温度 $T_a(t)$ 和空气水分浓度 $C_a(t)$ 沿用问题三的设定。附录 4 中的水分扩散系数以绝对温度为自变量，定义

$$T_{\mathrm{K}}(\xi, t) = T(\xi, t) + 273.15 \tag{72}$$

药材物性参数为

$$
\begin{cases}
\rho(C) = 760 + 90C \\
c_p(C) = 1850 + 2150\dfrac{C}{C+1} \\
k(C) = 0.12 + 0.20\dfrac{C}{C+1} \\
D\big(C, T_{\mathrm{K}}\big) = 4.2 \times 10^{-4} \mathrm{e}^{-\frac{0.30}{C}} \mathrm{e}^{-\frac{3850}{T_{\mathrm{K}}}}
\end{cases}
\tag{73}
$$

温度场模型

材料坐标下的温度场控制方程为

$$
\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{R^2(t)\xi}\frac{\partial}{\partial \xi}\left[\xi k(C)\frac{\partial T}{\partial \xi}\right], \qquad 0 < \xi < 1 \tag{74}
$$

初始温度均匀分布，满足

$$
T(\xi, 0) = 28\,^{\circ}\mathrm{C}, \qquad 0 \le \xi \le 1 \tag{75}
$$

中心处满足轴对称边界条件

$$
\left.\frac{\partial T}{\partial \xi}\right|_{\xi=0} = 0 \tag{76}
$$

由 $\partial/\partial r = R^{-1}(t)\partial/\partial\xi$ 可将表面对流换热边界转换为

$$
-\frac{k\big(C(1,t)\big)}{R(t)}\left.\frac{\partial T}{\partial \xi}\right|_{\xi=1} = h\left[T(1,t) - T_a(t)\right] \tag{77}
$$

水分浓度场模型

材料坐标下的水分浓度控制方程为

$$
\frac{\partial C}{\partial t} = \frac{1}{R^2(t)\xi}\frac{\partial}{\partial \xi}\left[\xi D\big(C, T_{\mathrm{K}}\big)\frac{\partial C}{\partial \xi}\right], \qquad 0 < \xi < 1 \tag{78}
$$

初始水分浓度均匀分布，满足

$$
C(\xi, 0) = 2.55\,\mathrm{kg/kg}, \qquad 0 \le \xi \le 1 \tag{79}
$$

中心处满足对称边界条件

$$
\left.\frac{\partial C}{\partial \xi}\right|_{\xi=0} = 0 \tag{80}
$$

表面对流传质边界为

$$-\frac{D\big(C(1,t),T_{\mathrm{K}}(1,t)\big)}{R(t)}\left.\frac{\partial C}{\partial \xi}\right|_{\xi=1}=h_m\left[C(1,t)-C_a(t)\right] \tag{81}$$

温度场和水分浓度场控制方程在 $\xi=0$ 处含有 $1/\xi$ 项。数值求解时，依据中心对称条件对轴心节点单独离散。

烘干终止条件

药材表面持续向空气传递水分，中心位置的水分浓度下降最慢。因此，以中心水分浓度首次达到目标值的时刻作为烘干完成时刻

$$t_{\mathrm{d}}=\min\left\{t>0\mid C(0,t)<0.15\right\} \tag{82}$$

综上所述，在本问题中建立了**考虑尺寸收缩的全程烘干热质传递模型：**

$$\begin{cases}\text{几何收缩关系} & \begin{cases} r=\xi R(t) \\ 0\le\xi\le 1 \end{cases} \\ \text{温度传递模型} & \begin{cases} \rho(C)c_p(C)\dfrac{\partial T}{\partial t}=\dfrac{1}{R^2(t)\xi}\dfrac{\partial}{\partial\xi}\left[\xi k(C)\dfrac{\partial T}{\partial\xi}\right] \\ T(\xi,0)=28 \\ \left.\dfrac{\partial T}{\partial\xi}\right|_{\xi=0}=0 \\ -\dfrac{k\big(C(1,t)\big)}{R(t)}\left.\dfrac{\partial T}{\partial\xi}\right|_{\xi=1}=h\left[T(1,t)-T_a(t)\right] \end{cases} \\ \text{水分迁移模型} & \begin{cases} \dfrac{\partial C}{\partial t}=\dfrac{1}{R^2(t)\xi}\dfrac{\partial}{\partial\xi}\left[\xi D\big(C,T_{\mathrm{K}}\big)\dfrac{\partial C}{\partial\xi}\right] \\ C(\xi,0)=2.55 \\ \left.\dfrac{\partial C}{\partial\xi}\right|_{\xi=0}=0 \\ -\dfrac{D\big(C(1,t),T_{\mathrm{K}}(1,t)\big)}{R(t)}\left.\dfrac{\partial C}{\partial\xi}\right|_{\xi=1}=h_m\left[C(1,t)-C_a(t)\right] \end{cases} \\ \text{烘干终止条件} & \left\{t_{\mathrm{d}}=\min\left\{t>0\mid C(0,t)<0.15\right\}\right. \end{cases} \tag{83}$$

5.4.2 问题四的模型求解

问题四的数值求解沿用问题三的有限体积离散、隐式求解和联合 Picard 迭代框架。其关键变化在于，药材半径随时间收缩，故以材料坐标组织计算，并在各计算时刻更新半径、物性参数及表面传递条件。

算法 4：考虑径向收缩的药材全程烘干模型求解

输入：烘房环境函数 $T_a(t)$、$C_a(t)$，半径函数 $R(t)$，药材初始状态及物性参数。
输出：考虑径向收缩后的烘干完成时刻 t_{d}。
Step 1 在材料坐标区间 $[0,1]$ 上划分固定网格，记

$$\xi_i = i\Delta\xi, \qquad \Delta\xi = \frac{1}{N_r}, \qquad t^n = n\Delta t \tag{84}$$

第 i 个节点在物理空间中的位置随半径变化：

$$r_i^n = \xi_i R(t^n) \tag{85}$$

温度场和水分浓度场的初值沿材料坐标均匀分布，即

$$T_i^0 = 28\,^{\circ}\mathrm{C}, \qquad C_i^0 = 2.55\,\mathrm{kg/kg} \tag{86}$$

Step 2 在时刻 t^{n+1}，根据 PCHIP 插值更新 $R(t^{n+1})$，并按前述环境条件确定 $T_a(t^{n+1})$ 和 $C_a(t^{n+1})$。以当前温度场和水分浓度场作为联合 Picard 迭代初值：

$$T_i^{(0)} = T_i^n, \qquad C_i^{(0)} = C_i^n \tag{87}$$

Step 3 在第 q 次迭代中，依据 $\boldsymbol{C}^{(q)}$ 更新 $\rho(C)$、$c_p(C)$ 和 $k(C)$；依据 $\boldsymbol{C}^{(q)}$ 及 $\boldsymbol{T}^{(q)}$ 更新水分扩散系数 $D(C,T_{\mathrm{K}})$。在材料坐标下离散温度场和水分浓度场方程，分别得到

$$\boldsymbol{A}_T\big(\boldsymbol{C}^{(q)}, R(t^{n+1})\big)\boldsymbol{T}^{(q+1)} = \boldsymbol{T}^n + \boldsymbol{b}_T\big(T_a(t^{n+1})\big) \tag{88}$$

$$\boldsymbol{A}_C\big(\boldsymbol{C}^{(q)}, \boldsymbol{T}_{\mathrm{K}}^{(q+1)}, R(t^{n+1})\big)\boldsymbol{C}^{(q+1)} = \boldsymbol{C}^n + \boldsymbol{b}_C\big(C_a(t^{n+1})\big) \tag{89}$$

两个方程组均采用 Thomas 追赶法求解。相较于问题三，矩阵系数中包含 $R^{-2}(t^{n+1})$，表面对流换热和传质边界中包含 $R^{-1}(t^{n+1})$，从而反映尺寸收缩对内部传递和表面交换的影响。由于采用材料坐标，干物质质量守恒，方程中不额外引入收缩导致的对流项。
Step 4 判断联合 Picard 迭代是否收敛。若满足

$$\max_i \left|T_i^{(q+1)} - T_i^{(q)}\right| < \varepsilon_T, \qquad \max_i \left|C_i^{(q+1)} - C_i^{(q)}\right| < \varepsilon_C \tag{90}$$

则得到时刻 t^{n+1} 的温度场和水分浓度场；否则，将迭代次数加 1，并重复 Step 3。
Step 5 对收敛后的中心水分浓度进行检验。当

$$C_0^{n+1} < 0.15\,\mathrm{kg/kg} \tag{91}$$

时，采用与问题三相同的局部二分法，在区间 $[t^n, t^{n+1}]$ 内定位阈值交叉时刻，得到烘干完成时刻 t_{d}；否则保存当前结果并继续计算。

计算完成后，对各计算时刻的水分浓度分布进行整理。考虑到药材半径随烘干过程

逐渐缩小，部分靠近初始表面的固定径向位置在后期不再位于药材内部。因此，选取全过程内均有效的距中心 0、0.5、1.0 cm 处水分浓度，并同时提取药材表面处的结果。每隔 6 h 的水分浓度及烘干完成时刻的结果如表 7所示。

表 **7**　药材烘干过程的水分浓度（kg/kg）

时间 t/h	离药材中心的距离 r/cm			
	0	0.5	1.0	药材表面
6	1.7177	1.5362	1.0216	0.4205
12	0.7378	0.6547	0.4083	0.1670
18	0.4090	0.3689	0.2399	0.0892
24	0.2853	0.2612	0.1791	0.0674
30	0.2265	0.2095	0.1494	0.0595
36	0.1929	0.1798	0.1317	0.0560
42	0.1713	0.1605	0.1200	0.0541
48	0.1562	0.1469	0.1116	0.0530
51.1749	0.1500	0.1413	0.1081	0.0526

由表 7可知，烘干至 51.1749h 时，药材中心水分浓度降至 0.1500kg/kg，其余径向位置均低于该阈值，满足题目规定的干燥要求。因此，考虑径向收缩后，药材所需烘干时间为 **51.1749**h。

5.4.3　问题四的结果分析

为直观展示药材收缩与水分迁移的时空演化关系，将计算结果映射至实际径向位置，绘制水分浓度热力图，如图 12所示。

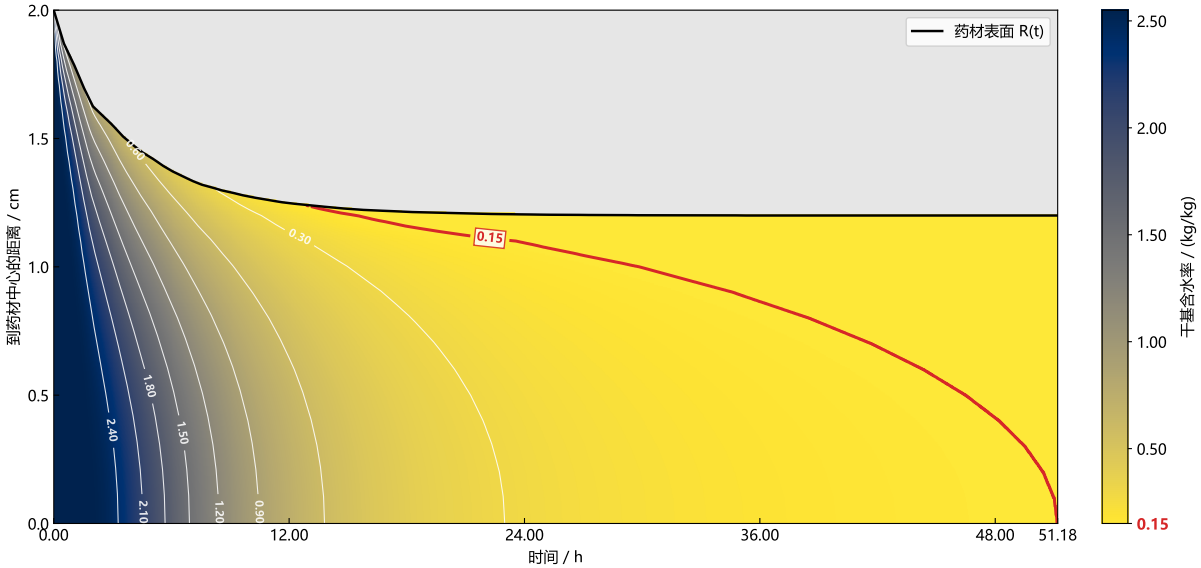


图 **12**　考虑径向收缩的药材水分浓度时空分布

由图 12可知，药材实际半径在烘干初期明显收缩，随后其变化逐渐减缓，热力图中的计算区域边界相应向中心移动。水分浓度始终表现为中心较高、表面较低的径向分布，低水分区域随烘干过程由表面逐步向内部扩展。

随着药材失水和尺寸收缩，水分浓度梯度逐渐减小，但中心区域仍是水分迁移的滞后部位。该结果表明，药材收缩与内部水分扩散在烘干过程中同步发生，材料坐标模型能够反映实际区域变化下的水分迁移特征。

5.5 问题四的模型检验

问题四在问题三的基础上进一步考虑药材烘干过程中的径向收缩，并通过材料坐标变换将时变计算区域映射至固定区间。为检验所建收缩模型及其数值离散方法的正确性，分别进行固定半径退化一致性检验和实际收缩条件下的水分质量平衡检验，结果如图 13所示。

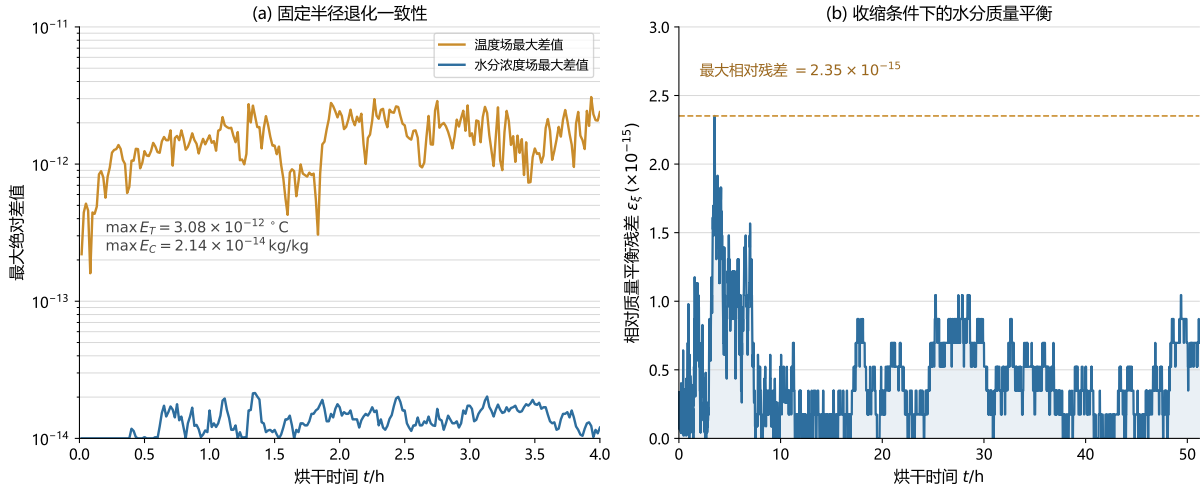


图 **13**　收缩模型退化一致性与水分质量平衡检验

固定半径退化一致性检验

将药材半径固定为初始半径：

$$R(t) \equiv R_0 \tag{92}$$

此时，材料坐标模型应退化为问题三的固定半径模型。在相同初始条件、边界条件、物性参数及离散参数下，定义两种模型的最大差值为

$$E_T(t) = \max_i \left| T_i^{\mathrm{mat}}(t) - T_i^{\mathrm{fix}}(t) \right|, \qquad E_C(t) = \max_i \left| C_i^{\mathrm{mat}}(t) - C_i^{\mathrm{fix}}(t) \right| \tag{93}$$

由图 13(a) 可知，在 0 至 4 h 内，$\max_t E_T(t) = 3.08 \times 10^{-12}$ °C，$\max_t E_C(t) = 2.14 \times 10^{-14}$ kg/kg

上述差值均处于浮点数舍入误差量级，说明当收缩效应消失时，材料坐标模型能够正确退化为固定半径模型；所建立的坐标变换关系及相应边界离散处理是正确的。**实际收缩条件下的水分收支平衡检验**

为检验药材收缩时的数值守恒性，在材料坐标下定义约化水分存量

$$\mathcal{M}_\xi(t) = \int_0^1 C(\xi, t)\, \xi \, \mathrm{d}\xi \tag{94}$$

由水分迁移方程、中心对称条件及表面对流传质条件可得

$$\frac{\mathrm{d}\mathcal{M}_{\xi}}{\mathrm{d}t}=\frac{1}{R^{2}(t)}\left[\xi D\big(C,T_{\mathrm{K}}\big)\frac{\partial C}{\partial \xi}\right]_{\xi=0}^{\xi=1}=-\frac{h_{m}}{R(t)}\left[C(1,t)-C_{a}(t)\right] \tag{95}$$

因此，定义从初始时刻至 t 时刻的累计约化水分逸出量为

$$\mathcal{Q}_{\xi}(t)=\int_{0}^{t}\frac{h_{m}}{R(\tau)}\left[C(1,\tau)-C_{a}(\tau)\right]\,\mathrm{d}\tau \tag{96}$$

水分收支平衡要求

$$\mathcal{M}_{\xi}(t)-\mathcal{M}_{\xi}(0)+\mathcal{Q}_{\xi}(t)=0 \tag{97}$$

据此定义相对水分收支残差

$$\varepsilon_{m,\xi}(t)=\frac{\left|\mathcal{M}_{\xi}(t)-\mathcal{M}_{\xi}(0)+\mathcal{Q}_{\xi}(t)\right|}{\mathcal{M}_{\xi}(0)} \tag{98}$$

数值计算中，约化水分存量由材料坐标网格上的控制体积求和得到，累计逸出量由各计算间隔的表面传质通量累加得到。由图 13(b) 可知，整个烘干过程中的最大相对残差为

$$\max_{t}\varepsilon_{m,\xi}(t)=2.35\times10^{-15} \tag{99}$$

该残差处于浮点数舍入误差量级，表明在半径变化条件下，数值结果仍满足所建材料坐标模型的水分收支关系，验证了收缩模型求解的守恒一致性。

六、 模型评估

6.1 模型优点

- 模型由预热阶段逐步扩展至变物性传递、目标水分浓度判定及药材径向收缩，能够对应不同阶段的主要烘干特征。
- 采用有限体积法、后向 Euler 格式、Thomas 追赶法和联合 Picard 迭代，对圆柱坐标下的非线性热质传递过程进行稳定求解。
- 通过插值敏感性、离散收敛性、二维轴对称复核、迭代收敛及水分质量平衡等检验，验证了模型结果的数值稳定性和物理一致性。

6.2 模型局限性

- 模型将药材视为均质轴对称圆柱体，主要考虑径向传递，并采用均匀收缩假设，未描述端面附近的局部传递差异及不同径向位置可能存在的非均匀收缩。
- 物性参数、水分扩散系数及表面换热、传质系数主要依据经验关系或固定取值确

定；附件环境数据结束后采用稳定环境条件，可能对长时间烘干过程的预测结果带来一定不确定性。

6.3 模型改进方向

- 后续可考虑收缩程度随局部水分浓度变化，建立非均匀收缩模型，以进一步描述药材表层与内部的尺寸差异。
- 可结合药材温度、水分浓度、尺寸及孔隙率的同步测量数据，对物性参数、扩散系数和表面传递系数进行标定，提高模型预测精度。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于相关文献检索、代码完善、图像绘制优化及论文语言润色，详细使用情况见支撑材料。

参考文献

[1] 陶文铨. 传热学 [M]. 5 版. 北京: 高等教育出版社, 2019.

[2] BIRD R B, STEWART W E, LIGHTFOOT E N. Transport phenomena[M]. 2nd ed. New York: Wiley, 2002.

[3] 陶文铨. 数值传热学 [M]. 2 版. 西安: 西安交通大学出版社, 2001.

[4] CRANK J. The mathematics of diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.

[5] KATEKAWA M E, SILVA M A. A review of drying models including shrinkage effects[J]. Drying Technology, 2006, 24(1): 5-20. DOI: 10.1080/07373930500538519.

附录

支撑材料列表	
源代码	问题 1_ 求解器.py 问题 1_ 热力图.py 问题 1_ 插值方法检验.py 问题 1_Bessel 解析检验.py 问题 2_ 求解器.py 问题 2_Picard 收敛检验.py 问题 3_ 求解器.py 问题 3_ 热力图.py 问题 4_ 求解器.py 问题 4_ 热力图.py 问题 4_ 收缩退化检验.py
结果表格	result1.xlsx result2.xlsx result3.xlsx result4.xlsx
图件	图 1 中药材热风烘干现场图.jpg 图 2 药材与烘房空气的换热换质示意图.pdf 图 3 附件一线性插值.pdf 图 4 算法 1 流程图.pdf 图 5 预热阶段药材内部温度和水分浓度的时空分布.pdf 图 6 线性插值与 PCHIP 插值下药材温度和水分浓度的计算差异.pdf 图 7 Bessel 解析解下网格与时间步长的收敛性检验.pdf 图 8 一维径向模型的二维轴对称复核.pdf 图 9 全烘干过程药材内部温度和水分浓度的时空分布.pdf 图 10_ 联合 Picard 迭代收敛性检验.pdf 图 11 全烘干过程药材内部水分浓度的时空分布.pdf 图 12 考虑径向收缩的药材水分浓度时空分布.pdf 图 13 收缩模型退化一致性与水分质量平衡检验.pdf
其他材料	AI 工具使用详情.pdf

A 问题一预热平衡阶段药材热质传递数值求解程序

```python
# -*- coding: utf-8 -*-
"""问题1预热平衡阶段求解器。

模型与物性来源：附录2；关键参数：h=25、h_m=8e-7、Δt=0.25 s、Δr=2.5×10  m。
输入数据：``../附件/附件1.xlsx``；输出位置：本目录 ``结果/``。
数值方法：有限体积 + 后向 Euler + Thomas 三对角 + Picard 迭代。
运行方式：在项目根目录执行 ``python 问题1/问题1_求解器.py``。
"""
import os
import numpy as np
from scipy.interpolate import PchipInterpolator
import openpyxl

# ==================== 路径 ====================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(SCRIPT_DIR)
RESULT_DIR = os.path.join(SCRIPT_DIR, '结果')
ATT1 = os.path.join(ROOT, '附件', '附件1.xlsx')

# ==================== 参数（附录2） ====================
R    = 0.02         # 半径 m
rho  = 820.0        # 密度 kg/m^3
cp   = 2600.0       # 比热容 J/(kg·K)
k    = 0.36         # 热传导系数 W/(m·K)
h    = 25.0         # 对流换热系数 W/(m^2·K)
hm   = 8e-7         # 对流传质系数 m/s
D0   = 7e-9         # 扩散系数前置因子 m^2/s
Da   = 0.89         # D = D0 * exp(-Da / C)

T0   = 28.0         # 初始温度 ℃
C0   = 2.55         # 初始水分浓度 kg/kg

t_end = 1800.0      # 求解总时长 s
dt    = 0.25        # 时间步长 s
N     = 800         # 径向网格数
dr    = R / N       # 与论文算法1一致（N_r=800、Δt=0.25 s、Δr=2.5×10  m）
alpha = k / (rho * cp)   # 热扩散率 m^2/s


def Dfun(C):
    """水分浓度扩散系数（附录2）：D = D0 * exp(-Da / C)"""
    return D0 * np.exp(-Da / C)


# ==================== 读取附件1 ====================
```

```python
def read_attachment1(path):
    wb = openpyxl.load_workbook(path, data_only=True)
    ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]   # 跳过表头
    t = np.array([r[0] for r in rows], dtype=float)
    T = np.array([r[1] for r in rows], dtype=float)
    C = np.array([r[2] for r in rows], dtype=float)
    return t, T, C

t_air, T_air, C_air = read_attachment1(ATT1)
T_air_interp = PchipInterpolator(t_air, T_air)
C_air_interp = PchipInterpolator(t_air, C_air)

# ==================== 网格与输出节点 ====================
r = np.arange(N + 1) * dr
n_steps = int(round(t_end / dt))

idx_table = [0, 200, 400, 600, 800]        # 0、0.5、1、1.5、2 cm
idx_full  = np.arange(0, N + 1, int(0.001 / dr))  # 每 0.1 cm，共 21 点
r_full    = idx_full * dr                 # 对应距离 m
r_table   = np.array(idx_table) * dr


# ==================== Thomas 算法 ====================
def thomas(a, b, c, d):
    """解三对角方程组 a[i]x[i-1]+b[i]x[i]+c[i]x[i+1]=d[i]，a[0]=c[N]=0"""
    a = a.copy(); b = b.copy(); c = c.copy(); d = d.copy()
    n = len(b)
    cp_ = np.zeros(n); dp_ = np.zeros(n)
    cp_[0] = c[0] / b[0]
    dp_[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp_[i - 1]
        cp_[i] = c[i] / m
        dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
    x = np.zeros(n)
    x[-1] = dp_[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp_[i] - cp_[i] * x[i + 1]
    return x


# ==================== 热方程系数（线性，常数矩阵） ====================
s   = alpha * dt / dr**2
A_h = 2 * N * h * dt / ((N - 0.25) * rho * cp * dr)   # 表面对流项系数
B_h = 2 * (N - 0.5) * s / (N - 0.25)

aT = np.zeros(N + 1); bT = np.zeros(N + 1); cT = np.zeros(N + 1)
```

```
bT[0] = 1 + 4 * s;  cT[0] = -4 * s                        # 中心
for i in range(1, N):
    aT[i] = -(i - 0.5) / i * s
    cT[i] = -(i + 0.5) / i * s
    bT[i] = 1 + 2 * s
aT[N] = -B_h;  bT[N] = 1 + A_h + B_h;  cT[N] = 0          # 表面(半格点)

# ==================== 水分方程表面对流项系数（常数） ====================
A_m = 2 * N * hm * dt / ((N - 0.25) * dr)


# ==================== 时间推进 ====================
T = np.full(N + 1, T0)
C = np.full(N + 1, C0)

T_full = np.zeros((n_steps, len(idx_full)))   # 每隔 0.25 s、每隔 0.1 cm 记录
C_full = np.zeros((n_steps, len(idx_full)))

times_table = [100, 300, 600, 900, 1200, 1500, 1800]
table_map = {t: i for i, t in enumerate(times_table)}
T_table = np.zeros((len(times_table), len(idx_table)))
C_table = np.zeros((len(times_table), len(idx_table)))

for n in range(n_steps):
    t_new = (n + 1) * dt

    # ---- 热方程（线性，直接求解一次） ----
    dT = T.copy()
    dT[N] += A_h * T_air_interp(t_new)
    T = thomas(aT, bT, cT, dT)

    # ---- 水分方程（Picard 迭代） ----
    C_new = C.copy()
    for it in range(50):
        C_prev = C_new.copy()
        C_face = 0.5 * (C_prev[:-1] + C_prev[1:])        # 界面水分浓度
        lam = Dfun(C_face) * dt / dr**2                  # 界面扩散数 _i

        aC = np.zeros(N + 1); bC = np.zeros(N + 1); cC = np.zeros(N + 1); dC = np.zeros(N + 1)
        bC[0] = 1 + 4 * lam[0];  cC[0] = -4 * lam[0];  dC[0] = C[0]     # 中心
        for i in range(1, N):
            aC[i] = -(i - 0.5) / i * lam[i - 1]
            cC[i] = -(i + 0.5) / i * lam[i]
            bC[i] = 1 + (i - 0.5) / i * lam[i - 1] + (i + 0.5) / i * lam[i]
            dC[i] = C[i]
        B_m = 2 * (N - 0.5) * lam[N - 1] / (N - 0.25)                   # 表面
        aC[N] = -B_m;  bC[N] = 1 + A_m + B_m;  cC[N] = 0
        dC[N] = C[N] + A_m * C_air_interp(t_new)
```

```
        C_new = thomas(aC, bC, cC, dC)
        if np.max(np.abs(C_new - C_prev)) < 1e-12:
            break
    C = C_new

    # ---- 记录 ----
    T_full[n] = T[idx_full]
    C_full[n] = C[idx_full]
    if t_new in table_map:
        j = table_map[t_new]
        T_table[j] = T[idx_table]
        C_table[j] = C[idx_table]

# ==================== 输出表1/表2（论文表格） ====================
def print_table(title, times, r_cm, data):
    print(f"\n{title}")
    header = "时间/s | " + " | ".join(f"{v:g}cm" for v in r_cm)
    print(header)
    print("-" * len(header))
    for i, t in enumerate(times):
        line = f"{t:>6d} | " + " | ".join(f"{data[i, j]:.4f}" for j in range(len(r_cm)))
        print(line)

r_table_cm = r_table * 100
print_table("表1  30分钟内药材的温度（℃）", times_table, r_table_cm, T_table)
print_table("表2  30分钟内药材的水分浓度（kg/kg）", times_table, r_table_cm, C_table)

# ==================== 验证 ====================
print("\n===== 验证 =====")
print(f"t=1800s 中心温度梯度 T[1]-T[0] = {T[1]-T[0]:.3e}（应 0）")
print(f"t=1800s 中心水分梯度 C[1]-C[0] = {C[1]-C[0]:.3e}（应 0）")
print(f"t=1800s 温度径向: 中心 T[0]={T[0]:.4f} → 表面 T[N]={T[N]:.4f}（表面应更热）")
print(f"t=1800s 水分径向: 中心 C[0]={C[0]:.4f} → 表面 C[N]={C[N]:.4f}（表面应更干）")
print(f"t=1800s 烘房温度={T_air_interp(1800):.4f}℃, 烘房水分={C_air_interp(1800):.5f} kg/kg")

# ==================== 写 result1.xlsx ====================
def write_result1(path, times, r_vals, T_full, C_full):
    wb = openpyxl.Workbook()

    ws = wb.active
    ws.title = "温度"
    ws.cell(1, 1, "时间\\到药材中心的距离")
    for j, rv in enumerate(r_vals):
        ws.cell(1, 2 + j, rv)
    for i, t in enumerate(times):
        ws.cell(2 + i, 1, t)
        for j in range(len(r_vals)):
```

```
            ws.cell(2 + i, 2 + j, round(float(T_full[i, j]), 4))

    ws2 = wb.create_sheet("水分浓度")
    ws2.cell(1, 1, "时间\\到药材中心的距离")
    for j, rv in enumerate(r_vals):
        ws2.cell(1, 2 + j, rv)
    for i, t in enumerate(times):
        ws2.cell(2 + i, 1, t)
        for j in range(len(r_vals)):
            ws2.cell(2 + i, 2 + j, round(float(C_full[i, j]), 4))

    wb.save(path)
    print(f"\n已写出 {path}（{len(times)} 行 × {len(r_vals)} 列 × 2 工作表）")

r_vals_round = [round(float(v) * 100, 1) for v in r_full]   # 0, 0.1, ..., 2.0（cm）
times_full = [(i + 1) * dt for i in range(n_steps)]
write_result1(os.path.join(RESULT_DIR, 'result1.xlsx'), times_full, r_vals_round, T_full,
    C_full)
```

B 问题二全烘干过程变物性热质传递数值求解程序

```
# -*- coding: utf-8 -*-
"""问题2全过程耦合传热传质求解器。

模型：耦合传热传质，变物性(附录3)，一维径向圆柱。
     = 650 + 128C
  cp = 1450 + 2736·C/(C+1)
  k  = 0.21 + 0.38·C/(C+1)
  D  = 2.4e-3·e^(-0.45/C)·e^(-3850/T)，T 单位 K
  h=25、hm=8e-7 沿用附录2
方法：后向 Euler 隐式 + 有限体积 + Thomas 三对角 + 耦合 Picard 迭代
输入：``../附件/附件1.xlsx``；输出：本目录 ``结果/``。
运行方式：在项目根目录执行 ``python 问题2/问题2_求解器.py``。
"""
import os
import numpy as np
import openpyxl
from scipy.interpolate import PchipInterpolator

# ==================== 路径 ====================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(SCRIPT_DIR)          # A题/
RESULT_DIR = os.path.join(ROOT, '问题2', '结果')
```

```
os.makedirs(RESULT_DIR, exist_ok=True)
ATT1 = os.path.join(ROOT, '附件', '附件1.xlsx')

# ==================== 参数 ====================
R    = 0.02          # 半径 m
h    = 25.0          # 对流换热系数 W/(m^2·K)
hm   = 8e-7          # 对流传质系数 m/s
T0   = 28.0          # 初始温度 ℃
C0   = 2.55          # 初始水分浓度 kg/kg
N    = 200           # 径向网格数
dr   = R / N
t_end = 10800.0      # 报告 3 h（3600*3）
dt   = 1.0           # 时间步长 s
n_steps = int(round(t_end / dt))


# ==================== 附录3 物性 ====================
def rho(C):    return 650.0 + 128.0 * C
def cp(C):     return 1450.0 + 2736.0 * C / (C + 1.0)
def kcond(C):  return 0.21 + 0.38 * C / (C + 1.0)
def Dfun(C, T_C):
    """水分浓度扩散系数，T_C 为摄氏温度（内部转开尔文）"""
    return 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / (T_C + 273.15))


# ==================== 读附件1 ====================
def read_att1(path):
    wb = openpyxl.load_workbook(path, data_only=True); ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]
    return (np.array([r[0] for r in rows], float),
            np.array([r[1] for r in rows], float),
            np.array([r[2] for r in rows], float))

t_air, T_air, C_air = read_att1(ATT1)
T_air_interp = PchipInterpolator(t_air, T_air)
C_air_interp = PchipInterpolator(t_air, C_air)


# ==================== 环境边界（预热段附件1 + 恒温段常数） ====================
def T_bc(t):
    return T_air_interp(t) if t <= 14400.0 else 49.97   # 平台均值（附件1 末2h平均）

def C_bc(t):
    return C_air_interp(t) if t <= 14400.0 else 0.0499


# ==================== Thomas ====================
def thomas(a, b, c, d):
```

```
    a = a.copy(); b = b.copy(); c = c.copy(); d = d.copy(); n = len(b)
    cp_ = np.zeros(n); dp_ = np.zeros(n)
    cp_[0] = c[0] / b[0]; dp_[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp_[i - 1]
        cp_[i] = c[i] / m
        dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
    x = np.zeros(n); x[-1] = dp_[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp_[i] - cp_[i] * x[i + 1]
    return x


# ==================== 热方程(变物性, 系数由 C 决定) ====================
def solve_heat(rhocp, k_face, T_old, tn):
    aT = np.zeros(N + 1); bT = np.zeros(N + 1); cT = np.zeros(N + 1); dT = np.zeros(N + 1)
    lam0 = k_face[0] * dt / (rhocp[0] * dr**2)                # 中心
    bT[0] = 1 + 4 * lam0; cT[0] = -4 * lam0; dT[0] = T_old[0]
    for i in range(1, N):
        lamL = k_face[i - 1] * dt / (rhocp[i] * dr**2)
        lamR = k_face[i] * dt / (rhocp[i] * dr**2)
        aT[i] = -(i - 0.5) / i * lamL
        cT[i] = -(i + 0.5) / i * lamR
        bT[i] = 1 + (i - 0.5) / i * lamL + (i + 0.5) / i * lamR
        dT[i] = T_old[i]
    Ah = 2 * N * h * dt / ((N - 0.25) * rhocp[N] * dr)          # 表面(半格点)
    lamN = k_face[N - 1] * dt / (rhocp[N] * dr**2)
    Bh = 2 * (N - 0.5) * lamN / (N - 0.25)
    aT[N] = -Bh; bT[N] = 1 + Ah + Bh; dT[N] = T_old[N] + Ah * T_bc(tn)
    return thomas(aT, bT, cT, dT)


# ==================== 水分方程（变物性, D 由 C、T 决定） ====================
def solve_mass(D_face, C_old, tn):
    lam = D_face * dt / dr**2
    aC = np.zeros(N + 1); bC = np.zeros(N + 1); cC = np.zeros(N + 1); dC = np.zeros(N + 1)
    bC[0] = 1 + 4 * lam[0]; cC[0] = -4 * lam[0]; dC[0] = C_old[0]   # 中心
    for i in range(1, N):
        aC[i] = -(i - 0.5) / i * lam[i - 1]
        cC[i] = -(i + 0.5) / i * lam[i]
        bC[i] = 1 + (i - 0.5) / i * lam[i - 1] + (i + 0.5) / i * lam[i]
        dC[i] = C_old[i]
    Am = 2 * N * hm * dt / ((N - 0.25) * dr)                    # 表面(半格点)
    Bm = 2 * (N - 0.5) * lam[N - 1] / (N - 0.25)
    aC[N] = -Bm; bC[N] = 1 + Am + Bm; dC[N] = C_old[N] + Am * C_bc(tn)
    return thomas(aC, bC, cC, dC)


```

```
# ==================== 时间推进（耦合 Picard 迭代） ====================
T = np.full(N + 1, T0)
C = np.full(N + 1, C0)

idx_full = np.arange(0, N + 1, 10)          # 0, 0.1, ..., 2.0 cm（21点）
idx_table = [0, 50, 100, 150, 200]          # 0, 0.5, 1, 1.5, 2 cm
r_full = idx_full * dr
r_table = np.array(idx_table) * dr

T_full = np.zeros((n_steps, len(idx_full)))
C_full = np.zeros((n_steps, len(idx_full)))

times_table = [1800, 3600, 5400, 7200, 9000, 10800]   # 0.5,1,1.5,2,2.5,3 h
tm = {t: i for i, t in enumerate(times_table)}
T_table = np.zeros((len(times_table), len(idx_table)))
C_table = np.zeros((len(times_table), len(idx_table)))

iter_hist = []

for n in range(n_steps):
    tn = (n + 1) * dt
    T_new = T.copy(); C_new = C.copy()
    for it in range(50):
        Tp = T_new.copy(); Cp = C_new.copy()
        # 物性（用上一迭代值）
        rhocp = rho(Cp) * cp(Cp)
        k_face = kcond(0.5 * (Cp[:-1] + Cp[1:]))
        # 热方程（线性，系数冻结）→ T_new
        T_new = solve_heat(rhocp, k_face, T, tn)
        # 水分方程采用当前浓度与新温度计算 D，并按 Gauss-Seidel 次序更新。
        Cf = 0.5 * (Cp[:-1] + Cp[1:])
        Tf = 0.5 * (T_new[:-1] + T_new[1:])
        D_face = Dfun(Cf, Tf)
        C_new = solve_mass(D_face, C, tn)
        # 收敛判据
        if np.max(np.abs(T_new - Tp)) < 1e-6 and np.max(np.abs(C_new - Cp)) < 1e-8:
            break
    T = T_new; C = C_new
    iter_hist.append(it)
    T_full[n] = T[idx_full]; C_full[n] = C[idx_full]
    if tn in tm:
        j = tm[tn]; T_table[j] = T[idx_table]; C_table[j] = C[idx_table]

# ==================== 输出表3/表4 ====================
def print_table(title, times_h, r_cm, data):
    print(f"\n{title}")
    header = "时间/h | " + " | ".join(f"{v:g}cm" for v in r_cm)
    print(header); print("-" * len(header))
```

```
    for i, th in enumerate(times_h):
        line = f"{th:>6.1f} | " + " | ".join(f"{data[i, j]:.4f}" for j in range(len(r_cm)))
        print(line)

times_h = [t / 3600.0 for t in times_table]
r_table_cm = r_table * 100
print_table("表3  3小时内药材的温度（℃）", times_h, r_table_cm, T_table)
print_table("表4  3小时内药材的水分浓度（kg/kg）", times_h, r_table_cm, C_table)

# ==================== 验证 ====================
print("\n===== 验证 =====")
print(f"t=3h 中心温度梯度 T[1]-T[0] = {T[1]-T[0]:.3e}（应 0）")
print(f"t=3h 中心水分梯度 C[1]-C[0] = {C[1]-C[0]:.3e}（应 0）")
print(f"t=3h 温度径向: 中心 {T[0]:.4f} → 表面 {T[N]:.4f}（表面应更热）")
print(f"t=3h 水分径向: 中心 {C[0]:.4f} → 表面 {C[N]:.4f}（表面应更干）")
print(f"Picard 迭代次数: 平均 {np.mean(iter_hist):.1f}, 最大 {np.max(iter_hist)}")
print(f"t=3h 烘房温度 {T_bc(10800):.4f}℃, 烘房水分 {C_bc(10800):.5f} kg/kg")

# 质量守恒（总含水量单调下降）
r = np.arange(N + 1) * dr
print(f"总含水量比(3h/初始) = {np.trapezoid(C * r, r) / np.trapezoid(C0 * r, r):.6f}（应<1）")

# ==================== 写 result2.xlsx ====================
r_vals = [round(float(v) * 100, 1) for v in r_full]
times = list(range(1, n_steps + 1))

wb = openpyxl.Workbook()
ws = wb.active; ws.title = "温度"
ws.cell(1, 1, "时间\\到药材中心的距离")
for j, rv in enumerate(r_vals):
    ws.cell(1, 2 + j, rv)
for i, t in enumerate(times):
    ws.cell(2 + i, 1, t)
    for j in range(len(r_vals)):
        ws.cell(2 + i, 2 + j, round(float(T_full[i, j]), 4)).number_format = '0.0000'
ws2 = wb.create_sheet("水分浓度")
ws2.cell(1, 1, "时间\\到药材中心的距离")
for j, rv in enumerate(r_vals):
    ws2.cell(1, 2 + j, rv)
for i, t in enumerate(times):
    ws2.cell(2 + i, 1, t)
    for j in range(len(r_vals)):
        ws2.cell(2 + i, 2 + j, round(float(C_full[i, j]), 4)).number_format = '0.0000'
wb.save(os.path.join(RESULT_DIR, 'result2.xlsx'))

# 表3/表4
wb = openpyxl.Workbook()
ws = wb.active; ws.title = "表3温度"
```

```
ws.cell(1, 1, "时间/h"); [ws.cell(1, 2 + j, f"{v:g}") for j, v in enumerate(r_table_cm)]
for i, th in enumerate(times_h):
    ws.cell(2 + i, 1, round(th, 1))
    for j in range(5):
        ws.cell(2 + i, 2 + j, round(float(T_table[i, j]), 4)).number_format = '0.0000'
ws2 = wb.create_sheet("表4水分")
ws2.cell(1, 1, "时间/h"); [ws2.cell(1, 2 + j, f"{v:g}") for j, v in enumerate(r_table_cm)]
for i, th in enumerate(times_h):
    ws2.cell(2 + i, 1, round(th, 1))
    for j in range(5):
        ws2.cell(2 + i, 2 + j, round(float(C_table[i, j]), 4)).number_format = '0.0000'
wb.save(os.path.join(RESULT_DIR, '表3表4.xlsx'))

print(f"\n已写出 问题2/结果/result2.xlsx 与 表3表4.xlsx")


```

C 问题三目标水分浓度下烘干完成时刻求解程序

```
# -*- coding: utf-8 -*-
"""问题3烘干时长求解器。

确定药材烘干所需时间（各处水分浓度 C < 0.15 kg/kg）。
模型：同问题2，耦合传热传质、变物性(附录3)、一维径向圆柱
边界：预热段(0-4h 附件1) + 恒温干燥段(>4h, T=49.97℃, C=0.0499，平台均值；噪声敏感性见 敏感性扫描.
    py)
方法：后向 Euler 隐式 + 有限体积 + Thomas + 耦合 Picard 迭代
判据：C(r=0,t) < 0.15（水分径向分布中心最高，全剖面单调性见 单调性检查.py）
输入：``../附件/附件1.xlsx``；输出：本目录 ``结果/``。
运行方式：在项目根目录执行 ``python 问题3/问题3_求解器.py``。
"""
import os
import numpy as np
import openpyxl
from scipy.interpolate import PchipInterpolator

# ==================== 路径 ====================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(SCRIPT_DIR)
RESULT_DIR = os.path.join(ROOT, '问题3', '结果')
os.makedirs(RESULT_DIR, exist_ok=True)
ATT1 = os.path.join(ROOT, '附件', '附件1.xlsx')

# ==================== 参数 ====================
R    = 0.02
h    = 25.0
```

```
hm   = 8e-7
T0   = 28.0
C0   = 2.55
N    = 200
dr   = R / N
dt   = 60.0               # 时间步长 s（与 result3 输出间隔一致）
C_target = 0.15           # 烘干判据
t_max = 500000.0          # 求解上限 ~5.8 天（提前达标则 break）
n_max = int(t_max / dt)
drying_bisection_steps = 16  # 首次跨越阈值后的固定二分次数


def rho(C):   return 650.0 + 128.0 * C
def cp(C):    return 1450.0 + 2736.0 * C / (C + 1.0)
def kcond(C): return 0.21 + 0.38 * C / (C + 1.0)
def Dfun(C, T_C):
    return 2.4e-3 * np.exp(-0.45 / C) * np.exp(-3850.0 / (T_C + 273.15))


# ==================== 读附件1 ====================
def read_att1(path):
    wb = openpyxl.load_workbook(path, data_only=True); ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]
    return (np.array([r[0] for r in rows], float),
            np.array([r[1] for r in rows], float),
            np.array([r[2] for r in rows], float))

t_air, T_air, C_air = read_att1(ATT1)
T_air_interp = PchipInterpolator(t_air, T_air)
C_air_interp = PchipInterpolator(t_air, C_air)

def T_bc(t): return T_air_interp(t) if t <= 14400.0 else 49.97   # 平台均值（附件1 末2h平均）
def C_bc(t): return C_air_interp(t) if t <= 14400.0 else 0.0499


# ==================== Thomas ====================
def thomas(a, b, c, d):
    a = a.copy(); b = b.copy(); c = c.copy(); d = d.copy(); n = len(b)
    cp_ = np.zeros(n); dp_ = np.zeros(n)
    cp_[0] = c[0] / b[0]; dp_[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp_[i - 1]
        cp_[i] = c[i] / m
        dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
    x = np.zeros(n); x[-1] = dp_[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp_[i] - cp_[i] * x[i + 1]
    return x
```

```
75
76
77  # ==================== 求解器（复用问题2 的耦合 Picard） ====================
78  def solve_heat(rhocp, k_face, T_old, tn, dt_step):
79      aT = np.zeros(N + 1); bT = np.zeros(N + 1); cT = np.zeros(N + 1); dT = np.zeros(N + 1)
80      lam0 = k_face[0] * dt_step / (rhocp[0] * dr**2)
81      bT[0] = 1 + 4 * lam0; cT[0] = -4 * lam0; dT[0] = T_old[0]
82      for i in range(1, N):
83          lamL = k_face[i - 1] * dt_step / (rhocp[i] * dr**2)
84          lamR = k_face[i] * dt_step / (rhocp[i] * dr**2)
85          aT[i] = -(i - 0.5) / i * lamL
86          cT[i] = -(i + 0.5) / i * lamR
87          bT[i] = 1 + (i - 0.5) / i * lamL + (i + 0.5) / i * lamR
88          dT[i] = T_old[i]
89      Ah = 2 * N * h * dt_step / ((N - 0.25) * rhocp[N] * dr)
90      lamN = k_face[N - 1] * dt_step / (rhocp[N] * dr**2)
91      Bh = 2 * (N - 0.5) * lamN / (N - 0.25)
92      aT[N] = -Bh; bT[N] = 1 + Ah + Bh; dT[N] = T_old[N] + Ah * T_bc(tn)
93      return thomas(aT, bT, cT, dT)
94
95
96  def solve_mass(D_face, C_old, tn, dt_step):
97      lam = D_face * dt_step / dr**2
98      aC = np.zeros(N + 1); bC = np.zeros(N + 1); cC = np.zeros(N + 1); dC = np.zeros(N + 1)
99      bC[0] = 1 + 4 * lam[0]; cC[0] = -4 * lam[0]; dC[0] = C_old[0]
100     for i in range(1, N):
101         aC[i] = -(i - 0.5) / i * lam[i - 1]
102         cC[i] = -(i + 0.5) / i * lam[i]
103         bC[i] = 1 + (i - 0.5) / i * lam[i - 1] + (i + 0.5) / i * lam[i]
104         dC[i] = C_old[i]
105     Am = 2 * N * hm * dt_step / ((N - 0.25) * dr)
106     Bm = 2 * (N - 0.5) * lam[N - 1] / (N - 0.25)
107     aC[N] = -Bm; bC[N] = 1 + Am + Bm; dC[N] = C_old[N] + Am * C_bc(tn)
108     return thomas(aC, bC, cC, dC)
109
110
111 def advance_one_step(T_old, C_old, tn, dt_step):
112     """从指定状态推进一个任意长度的时间步，用于阈值时间二分。"""
113     T_new = T_old.copy(); C_new = C_old.copy()
114     for it in range(50):
115         Tp = T_new.copy(); Cp = C_new.copy()
116         rhocp = rho(Cp) * cp(Cp)
117         k_face = kcond(0.5 * (Cp[:-1] + Cp[1:]))
118         T_new = solve_heat(rhocp, k_face, T_old, tn, dt_step)
119         Cf = 0.5 * (Cp[:-1] + Cp[1:])
120         Tf = 0.5 * (T_new[:-1] + T_new[1:])
121         D_face = Dfun(Cf, Tf)
122         C_new = solve_mass(D_face, C_old, tn, dt_step)
```

```
        if np.max(np.abs(T_new - Tp)) < 1e-6 and np.max(np.abs(C_new - Cp)) < 1e-8:
            break
    return T_new, C_new, it + 1


def bisect_drying_time(T_before, C_before, t_before, T_after, C_after, t_after):
    """从首次达标前的场开始固定二分 16 次。"""
    t_low, T_low, C_low = t_before, T_before.copy(), C_before.copy()
    t_high, T_high, C_high = t_after, T_after.copy(), C_after.copy()
    if C_low[0] < C_target or C_high[0] >= C_target:
        raise ValueError("二分端点未包围 C(r=0)=0.15。")

    iterations = 0
    while iterations < drying_bisection_steps:
        t_mid = 0.5 * (t_low + t_high)
        T_mid, C_mid, _ = advance_one_step(T_low, C_low, t_mid, t_mid - t_low)
        if C_mid[0] < C_target:
            t_high, T_high, C_high = t_mid, T_mid, C_mid
        else:
            t_low, T_low, C_low = t_mid, T_mid, C_mid
        iterations += 1
    return t_high, iterations


# ==================== 时间推进 ====================
T = np.full(N + 1, T0)
C = np.full(N + 1, C0)
idx_full = np.arange(0, N + 1, 10)          # 0..2cm 每0.1cm（21点）
idx_table = [0, 50, 100, 150, 200]          # 0,0.5,1,1.5,2 cm

C_full = np.zeros((n_max, len(idx_full)))   # 每60s、每0.1cm
T_full = np.zeros((n_max, len(idx_full)))

n_done = 0
t_dry = None
iter_hist = []
for n in range(n_max):
    tn = (n + 1) * dt
    T_before = T.copy(); C_before = C.copy()
    T_new = T.copy(); C_new = C.copy()
    for it in range(50):
        Tp = T_new.copy(); Cp = C_new.copy()
        rhocp = rho(Cp) * cp(Cp)
        k_face = kcond(0.5 * (Cp[:-1] + Cp[1:]))
        T_new = solve_heat(rhocp, k_face, T, tn, dt)
        Cf = 0.5 * (Cp[:-1] + Cp[1:])
        Tf = 0.5 * (T_new[:-1] + T_new[1:])
        D_face = Dfun(Cf, Tf)
```

```
        C_new = solve_mass(D_face, C, tn, dt)
        if np.max(np.abs(T_new - Tp)) < 1e-6 and np.max(np.abs(C_new - Cp)) < 1e-8:
            break
    T = T_new; C = C_new
    iter_hist.append(it + 1)
    C_full[n] = C[idx_full]; T_full[n] = T[idx_full]
    n_done = n + 1
    # 烘干判据：中心水分 < 0.15（首次满足）
    if C[0] < C_target:
        # 从首次达标前一个时间层开始，二分时间步长定位跨越时刻。
        t_dry, drying_bisection_iterations = bisect_drying_time(
            T_before, C_before, tn - dt, T_new, C_new, tn
        )
        break

if t_dry is None:
    print(f"警告：{t_max/3600:.1f} h 内未达到 C<0.15，中心 C={C[0]:.4f}")
    t_dry = n_done * dt

t_dry_h = t_dry / 3600.0
print(f"===== 问题3 结果 =====")
print(f"烘干所需时间：{t_dry:.1f} s = {t_dry_h:.4f} h")
print(f"结束时刻中心水分 C(r=0) = {C[0]:.4f} kg/kg（判据 <0.15）")
print(f"结束时刻表面水分 C(r=R) = {C[N]:.4f} kg/kg")
print(f"结束时刻温度：中心 {T[0]:.2f}℃ → 表面 {T[N]:.2f}℃")
print(f"Picard 迭代：平均 {np.mean(iter_hist):.1f} 次/步，最大 {np.max(iter_hist)} 次")

# ==================== 表5（每6h） ====================
r_table_cm = [0.0, 0.5, 1.0, 1.5, 2.0]
times_6h = list(range(6 * 3600, int(t_dry) + 1, 6 * 3600))   # 6,12,18,...h 至烘干结束
# 确保最后一行是烘干结束时刻
times_6h = [t for t in times_6h if t <= t_dry]
if times_6h[-1] < t_dry - 1:
    times_6h.append(int(round(t_dry)))

print(f"\n表5  药材烘干过程的水分浓度（kg/kg）")
header = "时间/h | " + " | ".join(f"{v:g}cm" for v in r_table_cm)
print(header); print("-" * len(header))
C_table_rows = []
for ts in times_6h:
    idx = min(int(round(ts / dt)) - 1, n_done - 1)
    row = C_full[idx][[0, 5, 10, 15, 20]]   # idx_table 对应 idx_full 的 0,5,10,15,20
    C_table_rows.append(row)
    line = f"{ts/3600:>6.1f} | " + " | ".join(f"{v:.4f}" for v in row)
    print(line)

# ==================== 写 result3.xlsx ====================
r_vals = [round(float(v) * 100, 1) for v in (idx_full * dr)]
```

```
times = list(range(60, int(n_done * dt) + 1, 60))   # 60,120,... 至烘干结束
n_out = len(times)

wb = openpyxl.Workbook()
ws = wb.active; ws.title = "水分浓度"
ws.cell(1, 1, "时间\\到药材中心的距离")
for j, rv in enumerate(r_vals):
    ws.cell(1, 2 + j, rv)
for i, t in enumerate(times):
    ws.cell(2 + i, 1, t)
    for j in range(len(r_vals)):
        ws.cell(2 + i, 2 + j, round(float(C_full[i, j]), 4)).number_format = '0.0000'
wb.save(os.path.join(RESULT_DIR, 'result3.xlsx'))

# 表5 存档
wb = openpyxl.Workbook()
ws = wb.active; ws.title = "表5水分"
ws.cell(1, 1, "时间/h"); [ws.cell(1, 2 + j, f"{v:g}") for j, v in enumerate(r_table_cm)]
for i, ts in enumerate(times_6h):
    ws.cell(2 + i, 1, round(ts / 3600.0, 2))
    for j in range(5):
        ws.cell(2 + i, 2 + j, round(float(C_table_rows[i][j]), 4)).number_format = '0.0000'
wb.save(os.path.join(RESULT_DIR, '表5.xlsx'))

print(f"\n已写出 问题3/结果/result3.xlsx（{n_out} 行）与 表5.xlsx")
```

D 问题四考虑径向收缩的药材烘干时间求解程序

```
# -*- coding: utf-8 -*-
"""问题4仿射收缩求解器。

考虑尺寸收缩确定烘干时长。
模型：材料(Lagrangian)坐标 xi=r/R(t)，仿射(均匀)收缩，一维轴对称
  水分：dC/dt = (1/(R^2 xi)) d/dxi( xi D dC/dxi )   —— 无对流项(干物质守恒)
  热：  rho*cp dT/dt = (1/(R^2 xi)) d/dxi( xi k dT/dxi )
物性：附录4（rho=760+90C，cp=1850+2150C/(C+1)，k=0.12+0.20C/(C+1)，
      D=4.2e-4*e^(-0.30/C)*e^(-3850/T)）
几何：R(t) 取自附件2（PCHIP 插值）；h=25、hm=8e-7 沿用附录2
判据：C(r=0)<0.15（中心最高，全剖面单调性同问题3已验证）。
输入：``../附件/附件1.xlsx``、``../附件/附件2.xlsx``；输出：本目录 ``结果/``。
运行方式：在项目根目录执行 ``python 问题4/问题4_求解器.py``。
"""
import os
import numpy as np
import openpyxl
from scipy.interpolate import PchipInterpolator
```

```

# ==================== 路径 ====================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(SCRIPT_DIR)
RESULT_DIR = os.path.join(ROOT, '问题4', '结果')
os.makedirs(RESULT_DIR, exist_ok=True)
ATT1 = os.path.join(ROOT, '附件', '附件1.xlsx')
ATT2 = os.path.join(ROOT, '附件', '附件2.xlsx')

# ==================== 参数 ====================
h    = 25.0
hm   = 8e-7
T0   = 28.0
C0   = 2.55
C_target = 0.15
N    = 200
dxi  = 1.0 / N
dt   = 60.0
t_max = 500000.0
nmax = int(t_max / dt)


# ==================== 附录4 物性 ====================
def rho(C):   return 760.0 + 90.0 * C
def cp(C):    return 1850.0 + 2150.0 * C / (C + 1.0)
def kcond(C): return 0.12 + 0.20 * C / (C + 1.0)
def Dfun(C, T_C):
    return 4.2e-4 * np.exp(-0.30 / C) * np.exp(-3850.0 / (T_C + 273.15))


# ==================== 读附件1（环境）、附件2（半径） ====================
def read_xlsx(path, ncol):
    wb = openpyxl.load_workbook(path, data_only=True)
    ws = wb.active
    rows = list(ws.iter_rows(values_only=True))[1:]
    return tuple(np.array([r[j] for r in rows], float) for j in range(ncol))

t_air, T_air, C_air = read_xlsx(ATT1, 3)
T_air_i = PchipInterpolator(t_air, T_air)
C_air_i = PchipInterpolator(t_air, C_air)

t_rad, R_rad = read_xlsx(ATT2, 2)
R_rad = R_rad / 100.0                       # cm -> m
R_pchip = PchipInterpolator(t_rad, R_rad)
R0 = float(R_rad[0]); R_min = float(R_rad[-1])


def T_bc(t): return T_air_i(t) if t <= 14400.0 else 49.97   # 平台均值（附件1 末2h平均）
```

```
def C_bc(t): return C_air_i(t) if t <= 14400.0 else 0.0499

def R_of(t):
    if t >= t_rad[-1]:
        return R_min
    return float(R_pchip(t))


# ==================== Thomas ====================
def thomas(a, b, c, d):
    a = a.copy(); b = b.copy(); c = c.copy(); d = d.copy(); n = len(b)
    cp_ = np.zeros(n); dp_ = np.zeros(n)
    cp_[0] = c[0] / b[0]; dp_[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp_[i - 1]
        cp_[i] = c[i] / m
        dp_[i] = (d[i] - a[i] * dp_[i - 1]) / m
    x = np.zeros(n); x[-1] = dp_[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp_[i] - cp_[i] * x[i + 1]
    return x


# ==================== 时间推进（材料坐标，耦合 Picard） ====================
T = np.full(N + 1, T0)
C = np.full(N + 1, C0)
c_center_prev = C0
t_dry = None
iter_hist = []
C_hist = []

for n in range(nmax):
    tn = (n + 1) * dt
    R = R_of(tn + 0.5 * dt)                 # 半时步半径
    R2 = R * R
    T_new = T.copy(); C_new = C.copy()
    for it in range(50):
        Tp = T_new.copy(); Cp = C_new.copy()
        rhocp = rho(Cp) * cp(Cp)
        k_face = kcond(0.5 * (Cp[:-1] + Cp[1:]))
        # ---- 热方程 ----
        aT = np.zeros(N + 1); bT = np.zeros(N + 1); cT = np.zeros(N + 1); dT = np.zeros(N + 1)
        l0 = k_face[0] * dt * N * N / (rhocp[0] * R2)
        bT[0] = 1 + 4 * l0; cT[0] = -4 * l0; dT[0] = T[0]
        for i in range(1, N):
            lL = k_face[i - 1] * dt * N * N / (rhocp[i] * R2)
            lR = k_face[i] * dt * N * N / (rhocp[i] * R2)
            aT[i] = -(i - 0.5) / i * lL
```

```
            cT[i] = -(i + 0.5) / i * lR
            bT[i] = 1 + (i - 0.5) / i * lL + (i + 0.5) / i * lR
            dT[i] = T[i]
        Ah = 2 * N * N * h * dt / ((N - 0.25) * rhocp[N] * R)
        lN = k_face[N - 1] * dt * N * N / (rhocp[N] * R2)
        Bh = 2 * (N - 0.5) * lN / (N - 0.25)
        aT[N] = -Bh; bT[N] = 1 + Ah + Bh; dT[N] = T[N] + Ah * T_bc(tn)
        T_new = thomas(aT, bT, cT, dT)
        # ---- 水分方程 ----
        Cf = 0.5 * (Cp[:-1] + Cp[1:]); Tf = 0.5 * (T_new[:-1] + T_new[1:])
        lam = Dfun(Cf, Tf) * dt * N * N / R2
        aC = np.zeros(N + 1); bC = np.zeros(N + 1); cC = np.zeros(N + 1); dC = np.zeros(N + 1)
        bC[0] = 1 + 4 * lam[0]; cC[0] = -4 * lam[0]; dC[0] = C[0]
        for i in range(1, N):
            aC[i] = -(i - 0.5) / i * lam[i - 1]
            cC[i] = -(i + 0.5) / i * lam[i]
            bC[i] = 1 + (i - 0.5) / i * lam[i - 1] + (i + 0.5) / i * lam[i]
            dC[i] = C[i]
        Am = 2 * N * N * hm * dt / ((N - 0.25) * R)
        Bm = 2 * (N - 0.5) * lam[N - 1] / (N - 0.25)
        aC[N] = -Bm; bC[N] = 1 + Am + Bm; dC[N] = C[N] + Am * C_bc(tn)
        C_new = thomas(aC, bC, cC, dC)
        if np.max(np.abs(T_new - Tp)) < 1e-6 and np.max(np.abs(C_new - Cp)) < 1e-8:
            break
    T = T_new; C = C_new
    iter_hist.append(it + 1)
    C_hist.append(C.copy())
    if C[0] < C_target:
        t_dry = (n + (C_target - c_center_prev) / (C[0] - c_center_prev)) * dt
        break
    c_center_prev = C[0]

if t_dry is None:
    print("警告: 未达标")
    t_dry = nmax * dt

print("===== 问题4 结果（仿射收缩 / 材料坐标, 附录4） =====")
print(f"烘干所需时间: {t_dry:.1f} s = {t_dry/3600.0:.4f} h")
print(f"结束时刻半径 R = {R_of(t_dry)*100:.4f} cm（初始 2.0 → 终态 {R_min*100:.4f} cm）")
print(f"结束时刻中心水分 C(r=0) = {C[0]:.4f}, 表面 C(R) = {C[N]:.4f}")
print(f"结束时刻温度: 中心 {T[0]:.2f}℃ → 表面 {T[N]:.2f}℃")
print(f"Picard 迭代: 平均 {np.mean(iter_hist):.1f} 次/步, 最大 {np.max(iter_hist)} 次")

# ==================== 输出表6 与 result4.xlsx（移动边界） ====================
n_done = len(C_hist)
xi_grid = np.arange(N + 1) / N
r_fixed = np.linspace(0.0, 0.02, 21)              # 0..2cm 共21点
times = [int((i + 1) * dt) for i in range(n_done)]
```

```

def C_at(rr, Cc, R):
    if rr > R + 1e-12:
        return None
    return float(np.interp(rr / R, xi_grid, Cc))

# result4.xlsx（固定物理距离列 + 药材表面列，材料外留空）
r_vals = [round(float(v) * 100, 1) for v in r_fixed]
wb = openpyxl.Workbook(); ws = wb.active; ws.title = "水分浓度"
ws.cell(1, 1, "时间\\到药材中心的距离")
for j, rv in enumerate(r_vals):
    ws.cell(1, 2 + j, rv)
ws.cell(1, 2 + len(r_vals), "药材表面")
for i, t in enumerate(times):
    R = R_of(t + 0.5 * dt); Cc = C_hist[i]
    ws.cell(2 + i, 1, t)
    for j, rr in enumerate(r_fixed):
        v = C_at(rr, Cc, R)
        if v is not None:
            ws.cell(2 + i, 2 + j, round(v, 4)).number_format = '0.0000'
    ws.cell(2 + i, 2 + len(r_vals), round(float(Cc[-1]), 4)).number_format = '0.0000'
wb.save(os.path.join(RESULT_DIR, 'result4.xlsx'))

# 表6（每6h，固定距离 0/0.5/1/1.5 cm + 药材表面）
r_tab = [0.0, 0.005, 0.01, 0.015]
print("\n表6  药材烘干过程的水分浓度（kg/kg）")
print("时间/h | " + " | ".join([f"{v*100:g}cm" for v in r_tab] + ["表面"]))
print("-" * 60)
for th in range(6, int(t_dry // 3600) + 1, 6):
    i = int(th * 3600 / dt) - 1
    if i >= n_done:
        break
    R = R_of(th * 3600); Cc = C_hist[i]
    row = []
    for rr in r_tab:
        v = C_at(rr, Cc, R)
        row.append("  --  " if v is None else f"{v:.4f}")
    row.append(f"{Cc[-1]:.4f}")
    print(f"{th:>6d} | " + " | ".join(row))

print(f"\n已写出 问题4/结果/result4.xlsx（{n_done} 行）")
```

E Bessel 解析检验

```
# -*- coding: utf-8 -*-
"""Bessel 解析解检验。
```

```
3
4  功能：以线性常 D 圆柱扩散解析解核对数值离散。模型来源：附录2线性检验。
5  关键参数：R、D、h_m、C0；输入数据：无外部文件；输出位置：标准输出及检验图。
6  运行方式：在项目根目录执行 ``python 问题1/问题1_Bessel解析检验.py``。
7  """
8  import numpy as np
9  from scipy.special import j0, j1
10 from scipy.optimize import brentq
11
12 # ============ 参数（线性情形） ============
13 R = 0.02; D = 5e-9; hm = 8e-7; C0 = 2.55; C_air = 0.0
14 Bi = hm * R / D
15 t_eval = 1800.0
16
17 # ============ 解析解（Bessel 级数） ============
18 def bessel_roots(Bi, n_terms, lam_max=200.0, ngrid=200000):
19     g = np.linspace(1e-6, lam_max, ngrid)
20     f = g * j1(g) - Bi * j0(g)
21     roots = []
22     for i in range(len(g) - 1):
23         if f[i] * f[i + 1] < 0:
24             roots.append(brentq(lambda x: x * j1(x) - Bi * j0(x), g[i], g[i + 1]))
25             if len(roots) >= n_terms:
26                 break
27     return np.array(roots)
28
29 lam = bessel_roots(Bi, 60)
30 A = 2.0 * j1(lam) / (lam * (j0(lam) ** 2 + j1(lam) ** 2))
31
32 def C_analytical(r, t):
33     return C_air + (C0 - C_air) * np.sum(A[:, None] * j0(np.outer(lam, r / R)) * np.exp(-(lam
       ** 2)[:, None] * D * t / R ** 2), axis=0)
34
35 # ============ 数值解（隐式有限体积，常数 D） ============
36 def thomas(a, b, c, d):
37     n = len(b); cp_ = np.zeros(n); dp_ = np.zeros(n)
38     cp_[0] = c[0] / b[0]; dp_[0] = d[0] / b[0]
39     for i in range(1, n):
40         m = b[i] - a[i] * cp_[i - 1]; cp_[i] = c[i] / m; dp_[i] = (d[i] - a[i] * dp_[i - 1]) /
       m
41     x = np.zeros(n); x[-1] = dp_[-1]
42     for i in range(n - 2, -1, -1): x[i] = dp_[i] - cp_[i] * x[i + 1]
43     return x
44
45 def C_numerical(N, dt):
46     dr = R / N; s = D * dt / dr**2
47     Ah = 2 * N * hm * dt / ((N - 0.25) * dr)
48     Bh = 2 * (N - 0.5) * s / (N - 0.25)
```

```
    aT = np.zeros(N + 1); bT = np.zeros(N + 1); cT = np.zeros(N + 1)
    bT[0] = 1 + 4 * s; cT[0] = -4 * s
    for i in range(1, N):
        aT[i] = -(i - 0.5) / i * s; cT[i] = -(i + 0.5) / i * s; bT[i] = 1 + 2 * s
    aT[N] = -Bh; bT[N] = 1 + Ah + Bh
    C = np.full(N + 1, C0)
    for _ in range(int(t_eval / dt)):
        d = C.copy(); d[N] += Ah * C_air
        C = thomas(aT, bT, cT, d)
    return C

r = np.linspace(0, R, 201)
C_exact = C_analytical(r, t_eval)

print(f"Bessel 解析解验证（D={D:.1e}, Bi={Bi:.3f}, t={t_eval}s, {len(lam)} 项级数）\n")
print("网格收敛（dt=1s）：")
for N in [100, 200, 400]:
    C_num = C_numerical(N, 1.0)
    rn = np.linspace(0, R, N + 1)
    err = np.max(np.abs(C_num - C_analytical(rn, t_eval)))
    print(f"  N={N}: max|误差| = {err:.3e} kg/kg")
print("\n时间步收敛（N=200）：")
for dt in [2.0, 1.0, 0.5]:
    C_num = C_numerical(200, dt)
    rn = np.linspace(0, R, 201)
    err = np.max(np.abs(C_num - C_analytical(rn, t_eval)))
    print(f"  dt={dt}: max|误差| = {err:.3e} kg/kg")
print("\n验证完成：误差随网格/时间步单调下降且量级达 1e-4 以下，即离散格式正确。")
```

F Picard 迭代收敛性检验

```
# -*- coding: utf-8 -*-
"""问题四模型检验：固定半径退化一致性与收缩条件下的质量平衡。

输入：同目录 ``问题4_求解器.py``；输出：同目录 ``图/``；运行方式：在项目根目录执行本文件。
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np


SOLVER_FILE = Path(__file__).resolve().parent / "问题4_求解器.py"
OUTPUT_DIR = Path(__file__).resolve().parent / "图"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```


def sci_math(value, digits=2):
    """数值 → mathtext 科学计数法（如 3.08×10^{-12}），避免 'e-12' 形式。"""
    if value == 0:
        return '0'
    exp = int(np.floor(np.log10(abs(value))))
    mant = value / 10 ** exp
    return rf'{mant:.{digits}f}\times10^{{{exp}}}'


def load_solver_definitions():
    source = SOLVER_FILE.read_text(encoding="utf-8")
    prefix = source.split("# ==================== 时间推进")[0]
    namespace = {"__file__": str(SOLVER_FILE), "__name__": "q4_solver_definitions"}
    exec(compile(prefix, str(SOLVER_FILE), "exec"), namespace)
    return namespace


def thomas_step_material(ns, temperature, concentration, current_time, radius):
    """问题四材料坐标下的联合 Picard 单次推进。"""
    n = ns["N"]
    dt = ns["dt"]
    h = ns["h"]
    hm = ns["hm"]
    rho = ns["rho"]
    cp = ns["cp"]
    kcond = ns["kcond"]
    d_fun = ns["Dfun"]
    t_bc = ns["T_bc"]
    c_bc = ns["C_bc"]
    updated_t = temperature.copy()
    updated_c = concentration.copy()

    for _ in range(50):
        prev_t = updated_t.copy()
        prev_c = updated_c.copy()
        rho_cp = rho(prev_c) * cp(prev_c)
        k_face = kcond(0.5 * (prev_c[:-1] + prev_c[1:]))
        radius_sq = radius**2

        a_t = np.zeros(n + 1)
        b_t = np.zeros(n + 1)
        c_t = np.zeros(n + 1)
        d_t = np.zeros(n + 1)
        l0 = k_face[0] * dt * n**2 / (rho_cp[0] * radius_sq)
        b_t[0] = 1.0 + 4.0 * l0
        c_t[0] = -4.0 * l0
```

```
        d_t[0] = temperature[0]
        for i in range(1, n):
            l_left = k_face[i - 1] * dt * n**2 / (rho_cp[i] * radius_sq)
            l_right = k_face[i] * dt * n**2 / (rho_cp[i] * radius_sq)
            a_t[i] = -(i - 0.5) / i * l_left
            c_t[i] = -(i + 0.5) / i * l_right
            b_t[i] = 1.0 - a_t[i] - c_t[i]
            d_t[i] = temperature[i]
        a_h = 2.0 * n**2 * h * dt / ((n - 0.25) * rho_cp[n] * radius)
        l_n = k_face[n - 1] * dt * n**2 / (rho_cp[n] * radius_sq)
        b_h = 2.0 * (n - 0.5) * l_n / (n - 0.25)
        a_t[n] = -b_h
        b_t[n] = 1.0 + a_h + b_h
        d_t[n] = temperature[n] + a_h * t_bc(current_time)
        updated_t = ns["thomas"](a_t, b_t, c_t, d_t)

        c_face = 0.5 * (prev_c[:-1] + prev_c[1:])
        t_face = 0.5 * (updated_t[:-1] + updated_t[1:])
        lam = d_fun(c_face, t_face) * dt * n**2 / radius_sq
        a_c = np.zeros(n + 1)
        b_c = np.zeros(n + 1)
        c_c = np.zeros(n + 1)
        d_c = np.zeros(n + 1)
        b_c[0] = 1.0 + 4.0 * lam[0]
        c_c[0] = -4.0 * lam[0]
        d_c[0] = concentration[0]
        for i in range(1, n):
            a_c[i] = -(i - 0.5) / i * lam[i - 1]
            c_c[i] = -(i + 0.5) / i * lam[i]
            b_c[i] = 1.0 - a_c[i] - c_c[i]
            d_c[i] = concentration[i]
        a_m = 2.0 * n**2 * hm * dt / ((n - 0.25) * radius)
        b_m = 2.0 * (n - 0.5) * lam[n - 1] / (n - 0.25)
        a_c[n] = -b_m
        b_c[n] = 1.0 + a_m + b_m
        d_c[n] = concentration[n] + a_m * c_bc(current_time)
        updated_c = ns["thomas"](a_c, b_c, c_c, d_c)

        if (
            np.max(np.abs(updated_t - prev_t)) < 1.0e-6
            and np.max(np.abs(updated_c - prev_c)) < 1.0e-8
        ):
            break
    return updated_t, updated_c


def thomas_step_fixed(ns, temperature, concentration, current_time, radius):
    """固定物理坐标下的对照模型单次推进。"""
```

```
    n = ns["N"]
    dt = ns["dt"]
    dr = radius / n
    h = ns["h"]
    hm = ns["hm"]
    rho = ns["rho"]
    cp = ns["cp"]
    kcond = ns["kcond"]
    d_fun = ns["Dfun"]
    t_bc = ns["T_bc"]
    c_bc = ns["C_bc"]
    updated_t = temperature.copy()
    updated_c = concentration.copy()

    for _ in range(50):
        prev_t = updated_t.copy()
        prev_c = updated_c.copy()
        rho_cp = rho(prev_c) * cp(prev_c)
        k_face = kcond(0.5 * (prev_c[:-1] + prev_c[1:]))

        a_t = np.zeros(n + 1)
        b_t = np.zeros(n + 1)
        c_t = np.zeros(n + 1)
        d_t = np.zeros(n + 1)
        l0 = k_face[0] * dt / (rho_cp[0] * dr**2)
        b_t[0] = 1.0 + 4.0 * l0
        c_t[0] = -4.0 * l0
        d_t[0] = temperature[0]
        for i in range(1, n):
            l_left = k_face[i - 1] * dt / (rho_cp[i] * dr**2)
            l_right = k_face[i] * dt / (rho_cp[i] * dr**2)
            a_t[i] = -(i - 0.5) / i * l_left
            c_t[i] = -(i + 0.5) / i * l_right
            b_t[i] = 1.0 - a_t[i] - c_t[i]
            d_t[i] = temperature[i]
        a_h = 2.0 * n * h * dt / ((n - 0.25) * rho_cp[n] * dr)
        l_n = k_face[n - 1] * dt / (rho_cp[n] * dr**2)
        b_h = 2.0 * (n - 0.5) * l_n / (n - 0.25)
        a_t[n] = -b_h
        b_t[n] = 1.0 + a_h + b_h
        d_t[n] = temperature[n] + a_h * t_bc(current_time)
        updated_t = ns["thomas"](a_t, b_t, c_t, d_t)

        c_face = 0.5 * (prev_c[:-1] + prev_c[1:])
        t_face = 0.5 * (updated_t[:-1] + updated_t[1:])
        lam = d_fun(c_face, t_face) * dt / dr**2
        a_c = np.zeros(n + 1)
        b_c = np.zeros(n + 1)
```

```
        c_c = np.zeros(n + 1)
        d_c = np.zeros(n + 1)
        b_c[0] = 1.0 + 4.0 * lam[0]
        c_c[0] = -4.0 * lam[0]
        d_c[0] = concentration[0]
        for i in range(1, n):
            a_c[i] = -(i - 0.5) / i * lam[i - 1]
            c_c[i] = -(i + 0.5) / i * lam[i]
            b_c[i] = 1.0 - a_c[i] - c_c[i]
            d_c[i] = concentration[i]
        a_m = 2.0 * n * hm * dt / ((n - 0.25) * dr)
        b_m = 2.0 * (n - 0.5) * lam[n - 1] / (n - 0.25)
        a_c[n] = -b_m
        b_c[n] = 1.0 + a_m + b_m
        d_c[n] = concentration[n] + a_m * c_bc(current_time)
        updated_c = ns["thomas"](a_c, b_c, c_c, d_c)

        if (
            np.max(np.abs(updated_t - prev_t)) < 1.0e-6
            and np.max(np.abs(updated_c - prev_c)) < 1.0e-8
        ):
            break
    return updated_t, updated_c


def fixed_radius_consistency(ns, end_time_s=14400.0):
    """冻结半径后，比较材料坐标与固定物理坐标结果。"""
    n = ns["N"]
    dt = ns["dt"]
    radius = ns["R0"]
    temp_material = np.full(n + 1, ns["T0"])
    moisture_material = np.full(n + 1, ns["C0"])
    temp_fixed = temp_material.copy()
    moisture_fixed = moisture_material.copy()
    steps = int(end_time_s / dt)
    times = np.empty(steps)
    error_t = np.empty(steps)
    error_c = np.empty(steps)

    for i in range(steps):
        current_time = (i + 1) * dt
        temp_material, moisture_material = thomas_step_material(
            ns, temp_material, moisture_material, current_time, radius
        )
        temp_fixed, moisture_fixed = thomas_step_fixed(
            ns, temp_fixed, moisture_fixed, current_time, radius
        )
        times[i] = current_time
```

```
        error_t[i] = np.max(np.abs(temp_material - temp_fixed))
        error_c[i] = np.max(np.abs(moisture_material - moisture_fixed))
    return times, error_t, error_c


def shrinking_mass_balance(ns, end_time_s=250000.0):
    """实际半径函数下，计算材料坐标的质量平衡残差。"""
    n = ns["N"]
    dt = ns["dt"]
    temp = np.full(n + 1, ns["T0"])
    moisture = np.full(n + 1, ns["C0"])
    radius_func = ns["R_of"]
    c_bc = ns["C_bc"]
    hm = ns["hm"]

    dxi = 1.0 / n
    node_index = np.arange(n + 1)
    weights = np.empty(n + 1)
    weights[0] = dxi**2 / 8.0
    weights[1:n] = node_index[1:n] * dxi**2
    weights[n] = (n - 0.25) * dxi**2 / 2.0
    initial_inventory = ns["C0"] / 2.0

    times = []
    residual = []
    outgoing = 0.0
    max_steps = int(end_time_s / dt)
    for i in range(max_steps):
        current_time = (i + 1) * dt
        radius = radius_func(current_time + 0.5 * dt)
        temp, moisture = thomas_step_material(ns, temp, moisture, current_time, radius)
        outgoing += hm / radius * (moisture[-1] - c_bc(current_time)) * dt
        inventory = moisture @ weights
        times.append(current_time)
        residual.append(abs(inventory - initial_inventory + outgoing) / initial_inventory)
        if moisture[0] < ns["C_target"]:
            break
    return np.asarray(times), np.asarray(residual)


def main():
    ns = load_solver_definitions()
    fixed_times, error_t, error_c = fixed_radius_consistency(ns)
    balance_times, residual = shrinking_mass_balance(ns)

    plt.rcParams.update(
        {
            "font.sans-serif": ["Microsoft YaHei", "SimHei", "Arial Unicode MS"],
```

```
            "axes.unicode_minus": False,
            "font.size": 11,
            "axes.labelsize": 12,
            "axes.titlesize": 12.5,
            "legend.fontsize": 9.5,
        }
    )
    fig, (ax_left, ax_right) = plt.subplots(
        1, 2, figsize=(12.5, 5.1), constrained_layout=True
    )

    # 左图：冻结半径时，变换模型应退化为固定物理坐标模型。
    plot_error_t = np.maximum(error_t, 1.0e-14)
    plot_error_c = np.maximum(error_c, 1.0e-14)
    ax_left.semilogy(fixed_times / 3600.0, plot_error_t, color="#C98C2B", lw=1.8, label="温度场最大差值")
    ax_left.semilogy(fixed_times / 3600.0, plot_error_c, color="#2E6E9E", lw=1.8, label="水分浓度场最大差值")
    ax_left.set(xlim=(0, 4.0), ylim=(1.0e-14, 1.0e-11))
    ax_left.set_xlabel(r"烘干时间 $t/\mathrm{h}$")
    ax_left.set_ylabel("最大绝对差值")
    ax_left.set_title("(a) 固定半径退化一致性")
    ax_left.grid(which="both", axis="y", color="#D9D9D9", linewidth=0.65)
    ax_left.spines[["top", "right"]].set_visible(False)
    ax_left.legend(loc="upper right")
    ax_left.text(
        0.20,
        2.3e-13,
        rf"$\max E_T={sci_math(np.max(error_t))}\,^\circ\mathrm{{C}}$" + "\n" + rf"$\max E_C={sci_math(np.max(error_c))}\,\mathrm{{kg/kg}}$",
        color="#444444",
    )

    # 右图：实际收缩下，材料坐标质量平衡残差。
    display_residual = residual * 1.0e15
    max_residual = float(np.max(residual))
    ax_right.plot(balance_times / 3600.0, display_residual, color="#2E6E9E", lw=1.7)
    ax_right.fill_between(balance_times / 3600.0, display_residual, 0.0, color="#2E6E9E", alpha=0.10)
    ax_right.axhline(max_residual * 1.0e15, color="#C98C2B", linestyle="--", linewidth=1.2)
    ax_right.set(
        xlim=(0, balance_times[-1] / 3600.0),
        ylim=(0, max(3.0, max_residual * 1.0e15 * 1.15)),
    )
    ax_right.set_xlabel(r"烘干时间 $t/\mathrm{h}$")
    ax_right.set_ylabel(r"相对质量平衡残差 $\varepsilon_\xi\,(\times10^{-15})$")
    ax_right.set_title("(b) 收缩条件下的水分质量平衡")
    ax_right.grid(axis="y", color="#D9D9D9", linewidth=0.65)
```

```
    ax_right.spines[["top", "right"]].set_visible(False)
    ax_right.text(0.04 * balance_times[-1] / 3600.0, 0.88 * ax_right.get_ylim()[1], rf"最大相对残差 $={sci_math(max_residual)}$", color="#9A661C")

    pdf_path = OUTPUT_DIR / "图12 收缩模型退化一致性与水分质量平衡检验.pdf"
    fig.savefig(pdf_path, bbox_inches="tight")
    plt.close(fig)
    print(f"max E_T = {np.max(error_t):.6e} °C")
    print(f"max E_C = {np.max(error_c):.6e} kg/kg")
    print(f"max mass residual = {max_residual:.6e}")
    print(pdf_path)


if __name__ == "__main__":
    main()
```

G 半径收缩退化检验

```
# -*- coding: utf-8 -*-
"""问题四模型检验：固定半径退化一致性与收缩条件下的质量平衡。

输入：同目录 ``问题4_求解器.py``；输出：同目录 ``图/``；运行方式：在项目根目录执行本文件。
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np


SOLVER_FILE = Path(__file__).resolve().parent / "问题4_求解器.py"
OUTPUT_DIR = Path(__file__).resolve().parent / "图"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)


def sci_math(value, digits=2):
    """数值 → mathtext 科学计数法（如 3.08×10^{-12}），避免 'e-12' 形式。"""
    if value == 0:
        return '0'
    exp = int(np.floor(np.log10(abs(value))))
    mant = value / 10 ** exp
    return rf'{mant:.{digits}f}\times10^{{{exp}}}'


def load_solver_definitions():
    source = SOLVER_FILE.read_text(encoding="utf-8")
```

```
    prefix = source.split("# ==================== 时间推进")[0]
    namespace = {"__file__": str(SOLVER_FILE), "__name__": "q4_solver_definitions"}
    exec(compile(prefix, str(SOLVER_FILE), "exec"), namespace)
    return namespace


def thomas_step_material(ns, temperature, concentration, current_time, radius):
    """问题四材料坐标下的联合 Picard 单次推进。"""
    n = ns["N"]
    dt = ns["dt"]
    h = ns["h"]
    hm = ns["hm"]
    rho = ns["rho"]
    cp = ns["cp"]
    kcond = ns["kcond"]
    d_fun = ns["Dfun"]
    t_bc = ns["T_bc"]
    c_bc = ns["C_bc"]
    updated_t = temperature.copy()
    updated_c = concentration.copy()

    for _ in range(50):
        prev_t = updated_t.copy()
        prev_c = updated_c.copy()
        rho_cp = rho(prev_c) * cp(prev_c)
        k_face = kcond(0.5 * (prev_c[:-1] + prev_c[1:]))
        radius_sq = radius**2

        a_t = np.zeros(n + 1)
        b_t = np.zeros(n + 1)
        c_t = np.zeros(n + 1)
        d_t = np.zeros(n + 1)
        l0 = k_face[0] * dt * n**2 / (rho_cp[0] * radius_sq)
        b_t[0] = 1.0 + 4.0 * l0
        c_t[0] = -4.0 * l0
        d_t[0] = temperature[0]
        for i in range(1, n):
            l_left = k_face[i - 1] * dt * n**2 / (rho_cp[i] * radius_sq)
            l_right = k_face[i] * dt * n**2 / (rho_cp[i] * radius_sq)
            a_t[i] = -(i - 0.5) / i * l_left
            c_t[i] = -(i + 0.5) / i * l_right
            b_t[i] = 1.0 - a_t[i] - c_t[i]
            d_t[i] = temperature[i]
        a_h = 2.0 * n**2 * h * dt / ((n - 0.25) * rho_cp[n] * radius)
        l_n = k_face[n - 1] * dt * n**2 / (rho_cp[n] * radius_sq)
        b_h = 2.0 * (n - 0.5) * l_n / (n - 0.25)
        a_t[n] = -b_h
        b_t[n] = 1.0 + a_h + b_h
```

```
        d_t[n] = temperature[n] + a_h * t_bc(current_time)
        updated_t = ns["thomas"](a_t, b_t, c_t, d_t)

        c_face = 0.5 * (prev_c[:-1] + prev_c[1:])
        t_face = 0.5 * (updated_t[:-1] + updated_t[1:])
        lam = d_fun(c_face, t_face) * dt * n**2 / radius_sq
        a_c = np.zeros(n + 1)
        b_c = np.zeros(n + 1)
        c_c = np.zeros(n + 1)
        d_c = np.zeros(n + 1)
        b_c[0] = 1.0 + 4.0 * lam[0]
        c_c[0] = -4.0 * lam[0]
        d_c[0] = concentration[0]
        for i in range(1, n):
            a_c[i] = -(i - 0.5) / i * lam[i - 1]
            c_c[i] = -(i + 0.5) / i * lam[i]
            b_c[i] = 1.0 - a_c[i] - c_c[i]
            d_c[i] = concentration[i]
        a_m = 2.0 * n**2 * hm * dt / ((n - 0.25) * radius)
        b_m = 2.0 * (n - 0.5) * lam[n - 1] / (n - 0.25)
        a_c[n] = -b_m
        b_c[n] = 1.0 + a_m + b_m
        d_c[n] = concentration[n] + a_m * c_bc(current_time)
        updated_c = ns["thomas"](a_c, b_c, c_c, d_c)

        if (
            np.max(np.abs(updated_t - prev_t)) < 1.0e-6
            and np.max(np.abs(updated_c - prev_c)) < 1.0e-8
        ):
            break
    return updated_t, updated_c


def thomas_step_fixed(ns, temperature, concentration, current_time, radius):
    """固定物理坐标下的对照模型单次推进。"""
    n = ns["N"]
    dt = ns["dt"]
    dr = radius / n
    h = ns["h"]
    hm = ns["hm"]
    rho = ns["rho"]
    cp = ns["cp"]
    kcond = ns["kcond"]
    d_fun = ns["Dfun"]
    t_bc = ns["T_bc"]
    c_bc = ns["C_bc"]
    updated_t = temperature.copy()
    updated_c = concentration.copy()
```

```
125
126     for _ in range(50):
127         prev_t = updated_t.copy()
128         prev_c = updated_c.copy()
129         rho_cp = rho(prev_c) * cp(prev_c)
130         k_face = kcond(0.5 * (prev_c[:-1] + prev_c[1:]))
131
132         a_t = np.zeros(n + 1)
133         b_t = np.zeros(n + 1)
134         c_t = np.zeros(n + 1)
135         d_t = np.zeros(n + 1)
136         l0 = k_face[0] * dt / (rho_cp[0] * dr**2)
137         b_t[0] = 1.0 + 4.0 * l0
138         c_t[0] = -4.0 * l0
139         d_t[0] = temperature[0]
140         for i in range(1, n):
141             l_left = k_face[i - 1] * dt / (rho_cp[i] * dr**2)
142             l_right = k_face[i] * dt / (rho_cp[i] * dr**2)
143             a_t[i] = -(i - 0.5) / i * l_left
144             c_t[i] = -(i + 0.5) / i * l_right
145             b_t[i] = 1.0 - a_t[i] - c_t[i]
146             d_t[i] = temperature[i]
147         a_h = 2.0 * n * h * dt / ((n - 0.25) * rho_cp[n] * dr)
148         l_n = k_face[n - 1] * dt / (rho_cp[n] * dr**2)
149         b_h = 2.0 * (n - 0.5) * l_n / (n - 0.25)
150         a_t[n] = -b_h
151         b_t[n] = 1.0 + a_h + b_h
152         d_t[n] = temperature[n] + a_h * t_bc(current_time)
153         updated_t = ns["thomas"](a_t, b_t, c_t, d_t)
154
155         c_face = 0.5 * (prev_c[:-1] + prev_c[1:])
156         t_face = 0.5 * (updated_t[:-1] + updated_t[1:])
157         lam = d_fun(c_face, t_face) * dt / dr**2
158         a_c = np.zeros(n + 1)
159         b_c = np.zeros(n + 1)
160         c_c = np.zeros(n + 1)
161         d_c = np.zeros(n + 1)
162         b_c[0] = 1.0 + 4.0 * lam[0]
163         c_c[0] = -4.0 * lam[0]
164         d_c[0] = concentration[0]
165         for i in range(1, n):
166             a_c[i] = -(i - 0.5) / i * lam[i - 1]
167             c_c[i] = -(i + 0.5) / i * lam[i]
168             b_c[i] = 1.0 - a_c[i] - c_c[i]
169             d_c[i] = concentration[i]
170         a_m = 2.0 * n * hm * dt / ((n - 0.25) * dr)
171         b_m = 2.0 * (n - 0.5) * lam[n - 1] / (n - 0.25)
172         a_c[n] = -b_m
```

```
        b_c[n] = 1.0 + a_m + b_m
        d_c[n] = concentration[n] + a_m * c_bc(current_time)
        updated_c = ns["thomas"](a_c, b_c, c_c, d_c)

        if (
            np.max(np.abs(updated_t - prev_t)) < 1.0e-6
            and np.max(np.abs(updated_c - prev_c)) < 1.0e-8
        ):
            break
    return updated_t, updated_c


def fixed_radius_consistency(ns, end_time_s=14400.0):
    """冻结半径后，比较材料坐标与固定物理坐标结果。"""
    n = ns["N"]
    dt = ns["dt"]
    radius = ns["R0"]
    temp_material = np.full(n + 1, ns["T0"])
    moisture_material = np.full(n + 1, ns["C0"])
    temp_fixed = temp_material.copy()
    moisture_fixed = moisture_material.copy()
    steps = int(end_time_s / dt)
    times = np.empty(steps)
    error_t = np.empty(steps)
    error_c = np.empty(steps)

    for i in range(steps):
        current_time = (i + 1) * dt
        temp_material, moisture_material = thomas_step_material(
            ns, temp_material, moisture_material, current_time, radius
        )
        temp_fixed, moisture_fixed = thomas_step_fixed(
            ns, temp_fixed, moisture_fixed, current_time, radius
        )
        times[i] = current_time
        error_t[i] = np.max(np.abs(temp_material - temp_fixed))
        error_c[i] = np.max(np.abs(moisture_material - moisture_fixed))
    return times, error_t, error_c


def shrinking_mass_balance(ns, end_time_s=250000.0):
    """实际半径函数下，计算材料坐标的质量平衡残差。"""
    n = ns["N"]
    dt = ns["dt"]
    temp = np.full(n + 1, ns["T0"])
    moisture = np.full(n + 1, ns["C0"])
    radius_func = ns["R_of"]
    c_bc = ns["C_bc"]
```

```
    hm = ns["hm"]

    dxi = 1.0 / n
    node_index = np.arange(n + 1)
    weights = np.empty(n + 1)
    weights[0] = dxi**2 / 8.0
    weights[1:n] = node_index[1:n] * dxi**2
    weights[n] = (n - 0.25) * dxi**2 / 2.0
    initial_inventory = ns["C0"] / 2.0

    times = []
    residual = []
    outgoing = 0.0
    max_steps = int(end_time_s / dt)
    for i in range(max_steps):
        current_time = (i + 1) * dt
        radius = radius_func(current_time + 0.5 * dt)
        temp, moisture = thomas_step_material(ns, temp, moisture, current_time, radius)
        outgoing += hm / radius * (moisture[-1] - c_bc(current_time)) * dt
        inventory = moisture @ weights
        times.append(current_time)
        residual.append(abs(inventory - initial_inventory + outgoing) / initial_inventory)
        if moisture[0] < ns["C_target"]:
            break
    return np.asarray(times), np.asarray(residual)


def main():
    ns = load_solver_definitions()
    fixed_times, error_t, error_c = fixed_radius_consistency(ns)
    balance_times, residual = shrinking_mass_balance(ns)

    plt.rcParams.update(
        {
            "font.sans-serif": ["Microsoft YaHei", "SimHei", "Arial Unicode MS"],
            "axes.unicode_minus": False,
            "font.size": 11,
            "axes.labelsize": 12,
            "axes.titlesize": 12.5,
            "legend.fontsize": 9.5,
        }
    )
    fig, (ax_left, ax_right) = plt.subplots(
        1, 2, figsize=(12.5, 5.1), constrained_layout=True
    )

    # 左图：冻结半径时，变换模型应退化为固定物理坐标模型。
    plot_error_t = np.maximum(error_t, 1.0e-14)
```

```
    plot_error_c = np.maximum(error_c, 1.0e-14)
    ax_left.semilogy(fixed_times / 3600.0, plot_error_t, color="#C98C2B", lw=1.8, label="温度场最大差值")
    ax_left.semilogy(fixed_times / 3600.0, plot_error_c, color="#2E6E9E", lw=1.8, label="水分浓度场最大差值")
    ax_left.set(xlim=(0, 4.0), ylim=(1.0e-14, 1.0e-11))
    ax_left.set_xlabel(r"烘干时间 $t/\mathrm{h}$")
    ax_left.set_ylabel("最大绝对差值")
    ax_left.set_title("(a) 固定半径退化一致性")
    ax_left.grid(which="both", axis="y", color="#D9D9D9", linewidth=0.65)
    ax_left.spines[["top", "right"]].set_visible(False)
    ax_left.legend(loc="upper right")
    ax_left.text(
        0.20,
        2.3e-13,
        rf"$\max E_T={sci_math(np.max(error_t))}\,^\circ\mathrm{{C}}$" + "\n" + rf"$\max E_C={sci_math(np.max(error_c))}\,\mathrm{{kg/kg}}$",
        color="#444444",
    )

    # 右图：实际收缩下，材料坐标质量平衡残差。
    display_residual = residual * 1.0e15
    max_residual = float(np.max(residual))
    ax_right.plot(balance_times / 3600.0, display_residual, color="#2E6E9E", lw=1.7)
    ax_right.fill_between(balance_times / 3600.0, display_residual, 0.0, color="#2E6E9E", alpha=0.10)
    ax_right.axhline(max_residual * 1.0e15, color="#C98C2B", linestyle="--", linewidth=1.2)
    ax_right.set(
        xlim=(0, balance_times[-1] / 3600.0),
        ylim=(0, max(3.0, max_residual * 1.0e15 * 1.15)),
    )
    ax_right.set_xlabel(r"烘干时间 $t/\mathrm{h}$")
    ax_right.set_ylabel(r"相对质量平衡残差 $\varepsilon_\xi\,(\times10^{-15})$")
    ax_right.set_title("(b) 收缩条件下的水分质量平衡")
    ax_right.grid(axis="y", color="#D9D9D9", linewidth=0.65)
    ax_right.spines[["top", "right"]].set_visible(False)
    ax_right.text(0.04 * balance_times[-1] / 3600.0, 0.88 * ax_right.get_ylim()[1], rf"最大相对残差 $={sci_math(max_residual)}$", color="#9A661C")

    pdf_path = OUTPUT_DIR / "图11_收缩模型退化一致性与质量平衡检验.pdf"
    fig.savefig(pdf_path, bbox_inches="tight")
    plt.close(fig)
    print(f"max E_T = {np.max(error_t):.6e} °C")
    print(f"max E_C = {np.max(error_c):.6e} kg/kg")
    print(f"max mass residual = {max_residual:.6e}")
    print(pdf_path)
```

```
312 if __name__ == "__main__":
313     main()
```